KTN Compendium

A “concise” review of TTM4100

Chapter 1: The Internet and Protocol Stack 5
1.1: What Is the Internet? 5
Internet Access Technologies 5
Physical Media 5
1.3: Packet Switching vs. Circuit Switching 6
Packet Switching 6
Circuit Switching 6
Network Structures (ISP Hierarchy) 6
1.4: Delay, Loss, and Throughput 6
1.5: The Protocol Stack 6
Chapter 2: The Application Layer 8
2.1: Application Architectures and Sockets 8
2.2: HTTP 8
HTTP Versions 8
HTTP Requests and Responses 9
Cookies 9
Web Caches (Proxy Servers) 10
Conditional GET 10
2.3: Email 10
2.4: DNS 11
DNS Hierarchy 11
DNS Caching and Resource Records 12
2.5: P2P File Distribution 13
Chapter 3: The Transport Layer 15
3.1: Overview 15
Multiplexing and Demultiplexing 15
3.2: UDP 15
Why Choose UDP over TCP? 15
UDP Checksum 15
3.3: Reliable Data Transfer (RDT) 15
Building Up to RDT 3.0 16
Pipelining: Go-Back-N and Selective Repeat 17
Go-Back-N (GBN) 17
Selective Repeat (SR) 18
3.4: TCP 19
Maximum Segment Size (MSS) 19
TCP Header 19
Sequence and ACK Numbers in Detail 20
TCP Timeout and Fast Retransmit 20

TCP Flow Control 20
TCP Connection Establishment (3-Way Handshake) 20
3.5: TCP Congestion Control 21
AIMD (Additive Increase, Multiplicative Decrease) 21
TCP Slow Start 21
TCP Tahoe vs. TCP Reno 21
TCP CUBIC 22
Delay-Based Congestion Control 22
TCP Fairness 23
3.6: QUIC 23
Chapter 4: The Network Layer: Data Plane 25
4.1: Overview 25
The Best-Effort Service Model 25
4.2: Inside a Router 25
Forwarding Approaches 26
Switching Fabric Technologies 26
Input Port Queuing and HOL Blocking 27
Output Port Queuing 27
Buffer Management (AQM) 28
Packet Scheduling 28
4.3: The Internet Protocol (IP) 29
IPv4 Datagram Format 29
IPv4 Addressing 29
Subnets and CIDR 30
DHCP (Dynamic Host Configuration Protocol) 30
NAT (Network Address Translation) 31
IPv6 32
IPv4-to-IPv6 Transition: Tunneling 33
Chapter 5: The Network Layer: Control Plane 34
5.1: Routing Algorithms 34
5.2: ICMP (Internet Control Message Protocol) 34
Chapter 6: The Link Layer 35
6.1: Overview and Services 35
6.2: Error Detection and Correction 35
Parity Checks 36
CRC (Cyclic Redundancy Check) 36
6.3: Multiple Access Protocols 37
Channel Partitioning Protocols (Collision Avoidance) 37
Random Access Protocols 38

Slotted ALOHA 38
CSMA and CSMA/CD 39
Taking-Turns Protocols 39
DOCSIS (Cable Access Networks) 39
6.4: Switched Local Area Networks 40
MAC Addresses 40
ARP (Address Resolution Protocol) 40
Ethernet Frame Structure 40
Switches 41
Switches vs. Routers 41
6.5: A Day in the Life of a Web Page Request 41
Chapter 7: Wireless and Mobile Networks 43
7.1: Wireless Network Taxonomy 43
7.2: Wireless Link Characteristics 43
SNR and BER 44
The Hidden Terminal Problem 44
7.3: Wi-Fi (802.11) 45
802.11 Architecture 45
Scanning and Association 46
CSMA/CA (Collision Avoidance) 46
RTS/CTS (Dealing with Hidden Terminals) 47
802.11 Frame Format 48
Mobility Within and Between Subnets 48
7.4: Cellular Networks: 4G and 5G 48
Cellular vs. Internet 48
4G LTE Architecture 48
LTE Link-Layer Protocol Stack 50
LTE Radio Access 50
LTE Network Attachment 51
Power Management 51
5G Cellular Networks 51
Chapter 8: Security 52
8.1: What Is Network Security? 52
8.2: Encryption 52
Classical Ciphers 52
Attack Scenarios 52
Block Ciphers 52
Public Key Encryption 53
8.3: Message Integrity and Authentication 53

Cryptographic Hash Functions 53
Message Authentication Code (MAC) 53
Digital Signatures 54
Public Key Certification 55
8.4: Securing Communication 56
Secure Email (Session Keys) 56
TLS (Transport Layer Security) 56
IPSec and VPNs 58
Wireless Security 58
8.5: Firewalls 58
Chapter 9: Multimedia Networking 60
9.1: Overview 60
9.2: Video Streaming 60
UDP Streaming 60
HTTP Streaming 60
9.3: Voice-over-IP (VoIP) 60
Playout Delay Strategies 60
Loss Recovery 61
RTP (Real-Time Transport Protocol) 61

Chapter 1: The Internet and Protocol Stack

1.1: What Is the Internet?

Applications run on **end systems** (also called hosts). The **socket interface** specifies how a program on one end system asks the Internet infrastructure to deliver data to a program on another end system.

A **protocol** defines the format and order of messages exchanged between systems, as well as the actions taken when a message is sent or received.

Internet Access Technologies

- **DSL (Digital Subscriber Line)**: Uses the existing telephone line. The telephone company acts as the ISP. A home modem converts digital signals to analogue; at the telco's central office, a multiplexer called a DSLAM converts them back.
- **Cable Internet**: Uses the TV cable infrastructure. This is a shared broadcast medium, meaning downstream data must travel through shared links.
- **FTTH (Fiber to the Home)**: A direct fiber connection from the central office (CO) to the home. In some setups, a shared splitter sits between homes. Two main architectures:
    - **AON (Active Optical Network)**: Essentially switched Ethernet.
    - **PON (Passive Optical Network)**: Uses Optical Network Terminators (ONTs) at each home and an Optical Line Terminator (OLT) at the CO.

Physical Media

- **Guided media** (e.g., copper, fiber) vs. **unguided media** (e.g., radio waves).

- **Coaxial cables** are concentric conductors; they act as a shared medium.
- **Terrestrial radio channels** are prone to shadow fading, multipath fading, and interference. They can be short-distance, local-area, or wide-area.

Packets travel at the full transmission rate of the link. If a packet is **L** bits long and the link rate is **R** bits/sec, the transmission time is ***L/R seconds***.

1.3: Packet Switching vs. Circuit Switching

Packet Switching

- Uses **store-and-forward transmission**: the entire packet must be received before it is forwarded.
- End-to-end delay for N links: ***N × L/R***.
- Packets may experience **queuing delays** in output buffers. If the buffer is full, **packet loss** occurs.
- Routers use a **forwarding table** to determine, based on the packet's destination IP, which router to forward to next.

Circuit Switching

In circuit switching, resources (transmission rate, buffers) are **reserved for the entire session**, unlike packet switching, where resources are shared on demand.

Two multiplexing approaches:

- **FDM (Frequency Division Multiplexing)**: The frequency spectrum is divided among connections, each getting a bandwidth slice (e.g., 4 kHz).
- **TDM (Time Division Multiplexing)**: Time is divided into frames and slots; each connection gets full bandwidth during its allocated slot.

Network Structures (ISP Hierarchy)

- **Structure 1:** A single global transit ISP serves all customer ISPs.
- **Structure 2:** Multiple global transit ISPs (Tier-1 ISPs).
- **Structure 3:** Multiple tiers of ISPs.
- **Structure 4:** Introduces Points of Presence (PoPs), multi-homing, peering, and Internet Exchange Points (IXPs).
    - **PoP**: A group of routers at a provider where customer ISPs connect.
    - **Multi-homing**: An access ISP connects to multiple provider ISPs.
    - **IXP**: A meeting point where ISPs can peer directly.
- **Structure 5:** Adds content-provider networks (e.g., Google's private network).

1.4: Delay, Loss, and Throughput

The total **nodal delay** has four components:

- **Processing delay**: Examining the header, checking for errors.
- **Queuing delay**: Waiting in the output buffer.
- **Transmission delay**: Pushing all bits onto the link (L/R).
- **Propagation delay**: Time for the signal to travel the physical medium.

Traffic intensity is defined as ***La/R***, where a = average packet arrival rate, L = packet size in bits, and R = link transmission rate. As traffic intensity approaches 1, queuing delay grows rapidly.

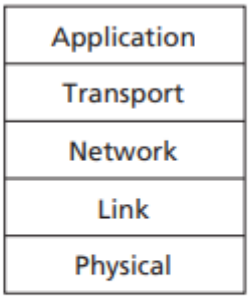
End-to-end delay across N nodes: $N \times (d_proc + d_trans + d_prop)$.

1.5: The Protocol Stack

The Internet uses a **five-layer protocol stack**:

1. **Application layer**: Where network applications and their protocols live (HTTP, SMTP, FTP). Distributed across end systems. Packet unit: ***message***.
2. **Transport layer**: Transports application messages between endpoints. TCP (reliable, connection-oriented, flow control) and UDP (connectionless, best-effort). Packet unit: ***segment***.
3. **Network layer**: Moves datagrams from source to destination host. The IP protocol operates here. Packet unit: ***datagram***.
4. **Link layer**: Routes datagrams through a series of routers (Ethernet, WiFi, DOCSIS). Packet unit: ***frame***.
5. **Physical layer**: Moves individual bits over the physical medium (twisted-pair copper, fiber optics, etc.).

Each layer **encapsulates** the data from the layer above by adding its own header.



Five-layer Internet protocol stack

Chapter 2: The Application Layer

2.1: Application Architectures and Sockets

Two main architectures:

- **Client-Server (CS)**: An always-on server responds to requests from clients.
- **Peer-to-Peer (P2P)**: No dedicated server; hosts communicate directly with each other.

A **socket** is the interface between the application layer and the transport layer. It's often referred to as the application's API to the network.

The **IP address** specifies the destination host; the **port number** specifies the receiving process on that host.

Transport-layer services can provide:

- Reliable data transfer
- Throughput guarantees
- Timing guarantees
- Security

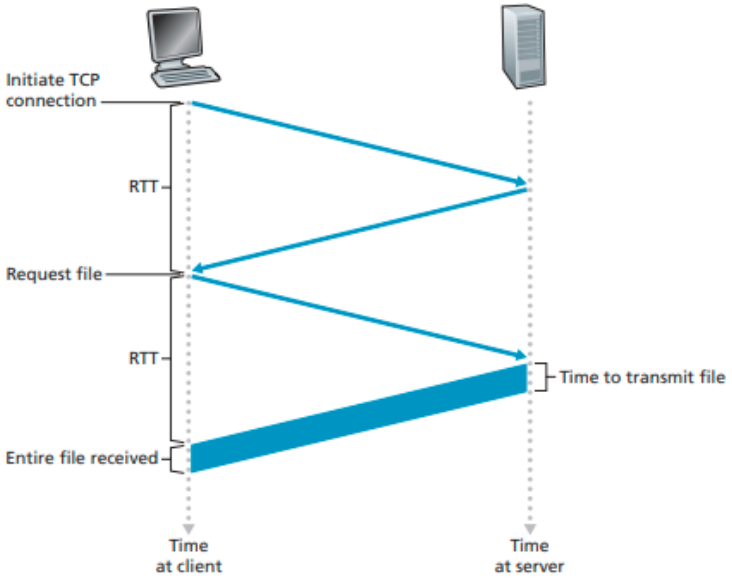
TCP offers connection-oriented, reliable data transfer with congestion control. **UDP** offers a minimal, connectionless, unreliable service.

2.2: HTTP

HTTP is the application-layer protocol for the Web. A web page consists of **objects** (HTML files, images, etc.), each addressable by a URL with two components: the **hostname** and the **object path**. HTTP is a **stateless protocol**, it does not retain information about clients between requests.

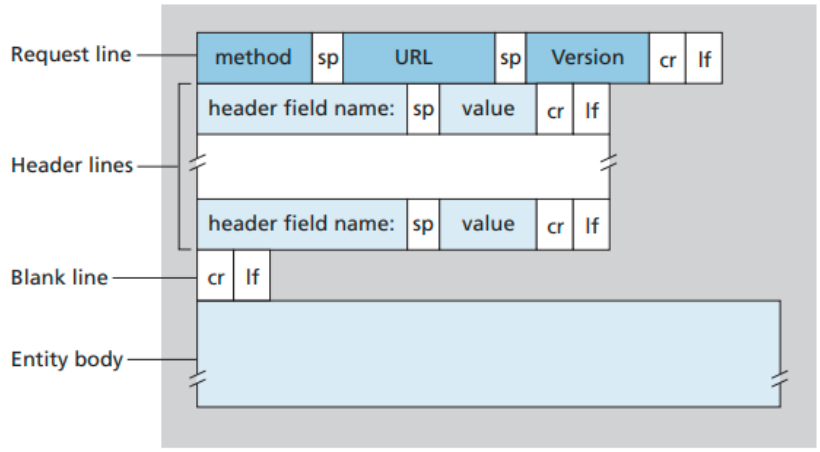
HTTP Versions

- **HTTP/1.0 (Non-persistent)**: A new TCP connection opens for each object. Costs **2 RTTs per object** (one to set up TCP, one to request and receive).
- **HTTP/1.1 (Persistent)**: Multiple objects can be sent over a single TCP connection.
- **HTTP/2**: Introduces **framing** to solve head-of-line (HOL) blocking. Servers can prioritize objects, push unrequested objects, and break large objects into interleaved frames.
- **HTTP/3**: Runs over **QUIC** (on top of UDP). Adds per-object error and congestion control.



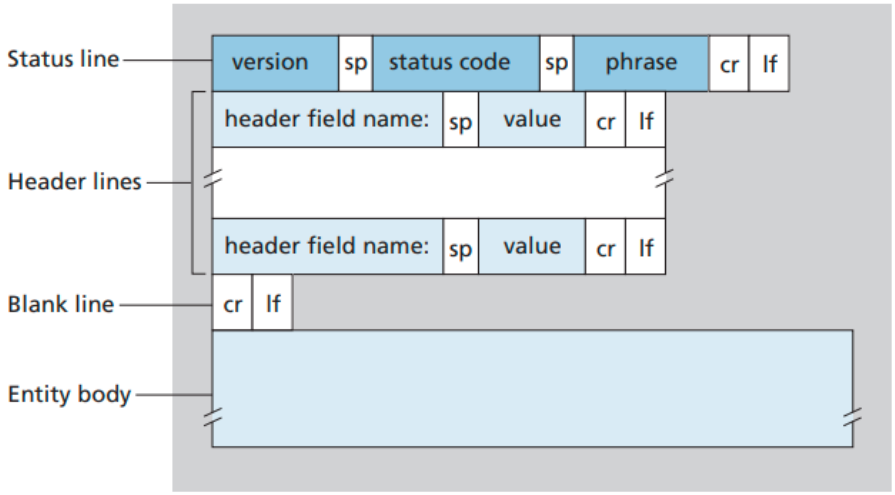
HTTP Requests and Responses

An HTTP request includes a method (**GET, POST, HEAD, PUT**) and headers. The **Connection** header (open/close/keep-alive) determines the connection type.



gure 2.8 ♦ General format of an HTTP request message

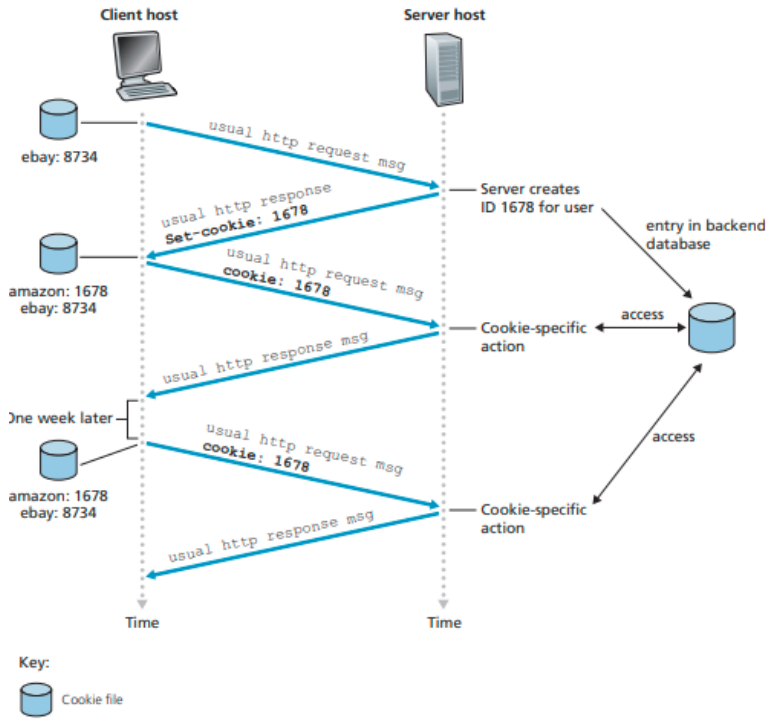
An HTTP response includes a **status code** in its status line (e.g., 200 OK, 301 Moved Permanently, 404 Not Found).



gure 2.9 ♦ General format of an HTTP response message

Cookies

Since HTTP is stateless, servers use **cookies** to maintain user state. A cookie is an identifier sent in the request/response headers. The server stores information about the user keyed to that identifier.



Web Caches (Proxy Servers)

A **web cache** stores copies of recently requested objects. When a user requests an object, the cache can serve it directly without contacting the origin server. Since LAN throughput is typically much higher than the access link to the Internet, caching can be very effective, it reduces traffic on the entire network, speeding up retrieval of both cached and non-cached objects.

Conditional GET

Goal: avoid sending an object the client already has. The client includes an ***If-Modified-Since: <date>*** header. If the object hasn't changed, the server responds with **304 Not Modified** (no body). If it has changed, the server sends the updated object with a normal **200 OK**.

2.3: Email

The email system has three main components: **user agents**, **mail servers**, and **SMTP (Simple Mail Transfer Protocol)**.

Key facts about SMTP:

- Runs on TCP, port 25.
- Uses 7-bit ASCII encoding (a legacy constraint).
- Is a **push protocol** (the sender pushes email to the receiver's mail server). This contrasts with HTTP, which is a pull protocol.
- Mail is sent directly between mail servers, no intermediaries.
- Uses **persistent connections**: multiple messages can be sent over the same TCP connection.

SMTP handshake (after TCP connection):

6. Server sends: *220 <server name>*
7. Client responds: *HELO <client name>*
8. Server responds: *250 Hello <client name>*

Each message begins with **MAIL FROM:** and the session ends with **QUIT**. To retrieve messages, recipients use **HTTP** or **IMAP (Internet Mail Access Protocol)**.

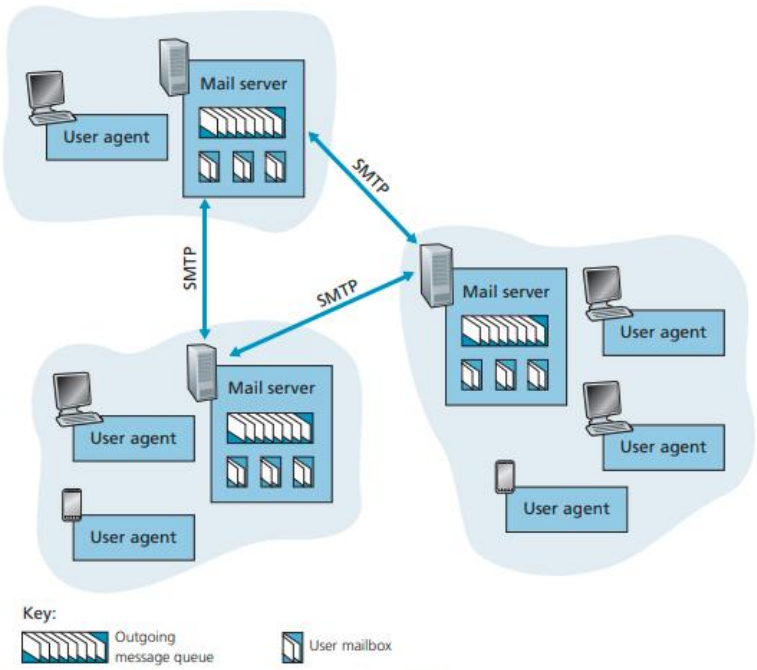


Figure 2.14 ♦ A high-level view of the Internet e-mail system

2.4: DNS

The **Domain Name System (DNS)** translates human-readable names (e.g., google.com) into IP addresses. It is:

- A **distributed database** implemented as a hierarchy of DNS servers.
- An **application-layer protocol** running over UDP on port 53.

Why is DNS distributed and hierarchical?

- Avoids a single point of failure.
- Distributes the load across many servers.
- Allows returning different IPs for load balancing.
- Reduces RTT by having servers closer to users.
- Easier to maintain.

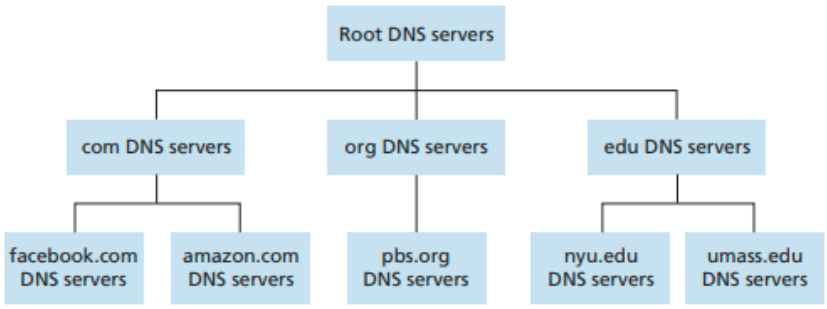
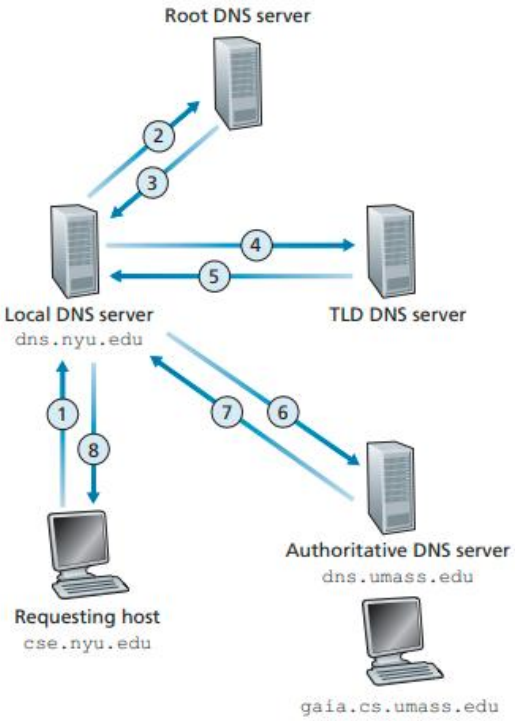


Figure 2.17 ♦ Portion of the hierarchy of DNS servers

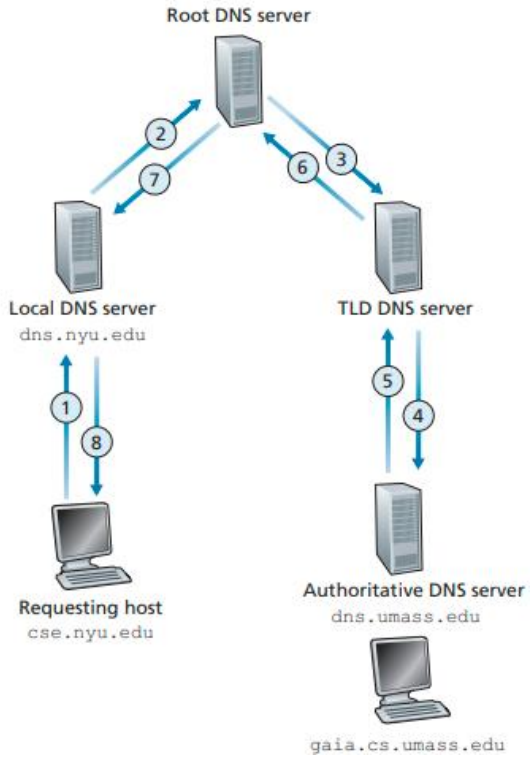
DNS Hierarchy

DNS servers are organized into: **root servers**, **top-level domain (TLD) servers**, and **authoritative DNS servers**. Queries can be:

- **Iterative**: The contacted server replies with the address of the next server to ask ("I don't know, but ask this server").



- **Recursive**: The contacted server takes responsibility for resolving the name fully. Rarely used in practice due to the heavy burden on upper-level servers.



DNS Caching and Resource Records

DNS caching improves response time. Cached entries eventually expire based on their **TTL (Time to Live)**.

DNS stores **resource records (RRs)** as tuples of the form (Name, Value, Type, TTL):

- **Type = A**: Name is a hostname, Value is an IP address.
- **Type = NS**: Name is a domain, Value is the hostname of an authoritative DNS server.
- **Type = CNAME**: Name is an alias, Value is the canonical hostname.
- **Type = MX**: Like CNAME but for mail servers.

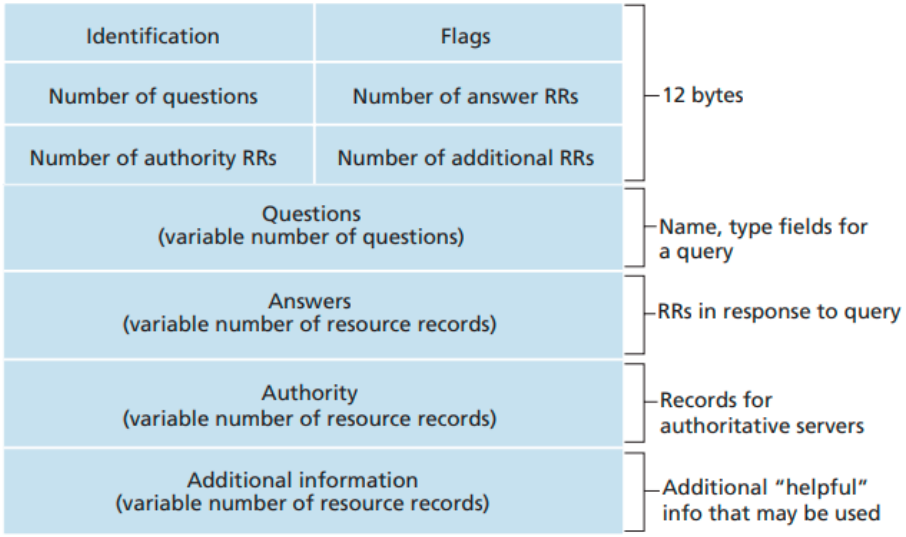


Figure 2.21 ♦ DNS message format

2.5: P2P File Distribution

In a **client-server** model, the server must upload $N \times F$ bits to distribute a file of size F to N clients. The minimum distribution time is:

$$D_{cs} = \max\{ N \times F / U_s , F / D_{min} \}$$

where U_s is the server's upload speed and D_min is the slowest client's download speed.

In a **P2P** model, peers also contribute upload capacity. The minimum distribution time is:

$$D_{P2P} = \max\{ F/U_s , F/D_{min} , N \times F/(U_s + \Sigma u_i) \}$$

The key insight: P2P scales much better because total upload capacity grows with the number of peers.

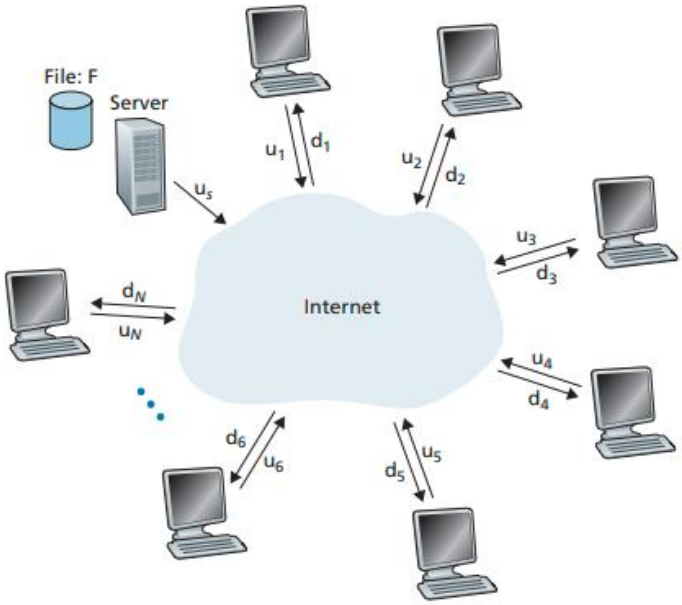


Figure 2.22 ♦ An illustrative file distribution problem

Chapter 3: The Transport Layer

3.1: Overview

The transport layer provides logical communication between **processes** (not just hosts, that's the network layer's job). Transport-layer protocols run on end systems, not routers.

Key services:

- **Multiplexing/Demultiplexing**
- Reliable data transfer
- Flow control
- Congestion control

TCP (Transmission Control Protocol) is connection-oriented and provides all four services. **UDP** (User Datagram Protocol) is connectionless and provides essentially none of them.

Multiplexing and Demultiplexing

- **Demultiplexing**: The transport layer delivers incoming data to the correct socket (a host may have multiple sockets for different processes).
- **Multiplexing**: The transport layer gathers data from multiple sockets, adds headers, and passes segments down to the network layer.

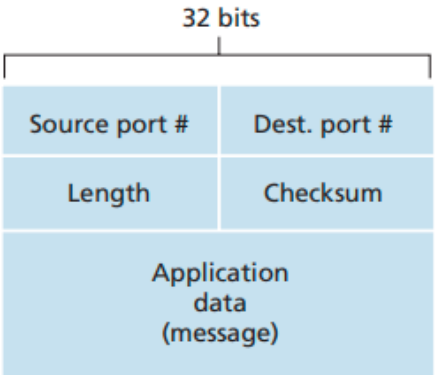
3.2: UDP

UDP is connectionless, provides no ordering guarantees, and operates on a best-effort basis.

Protocols that use UDP include: streaming multimedia (loss-tolerant, rate-sensitive), DNS, SNMP, and HTTP/3. If reliability is needed over UDP, it must be implemented at the application layer.

Why Choose UDP over TCP?

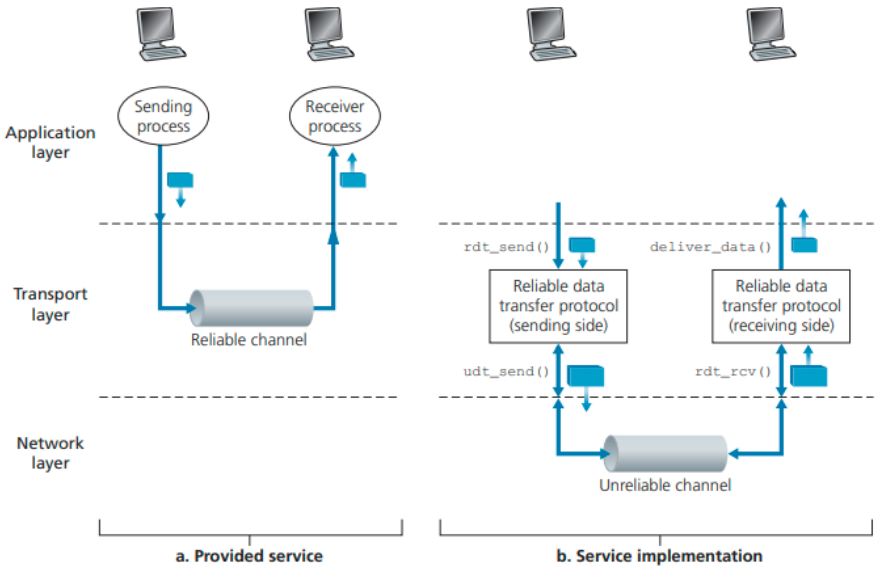
- **Finer application-level control** over what data is sent and when.
- **No congestion-control throttling**, data can be sent at any rate.
- **No connection setup delay** (this is why DNS uses UDP).
- **No connection state** to maintain, servers can support more concurrent users.
- **Smaller header overhead** (8 bytes vs. TCP's 20+ bytes).



UDP Checksum

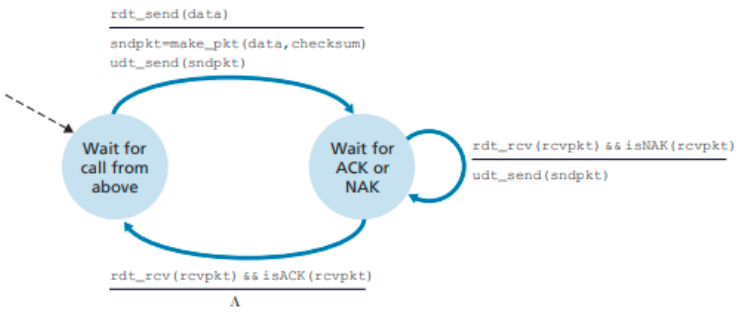
The checksum is the **1's complement of the sum of all 16-bit words** in the segment (with wraparound). The receiver sums all 16-bit words including the checksum, if the result is all 1s (0xFFFF), no errors are detected.

3.3: Reliable Data Transfer (RDT)

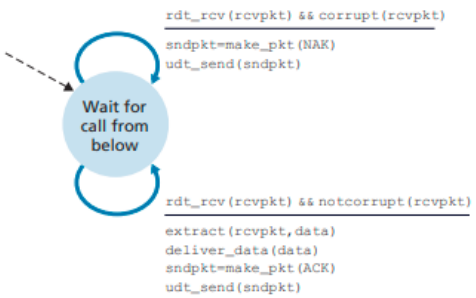


Building Up to RDT 3.0

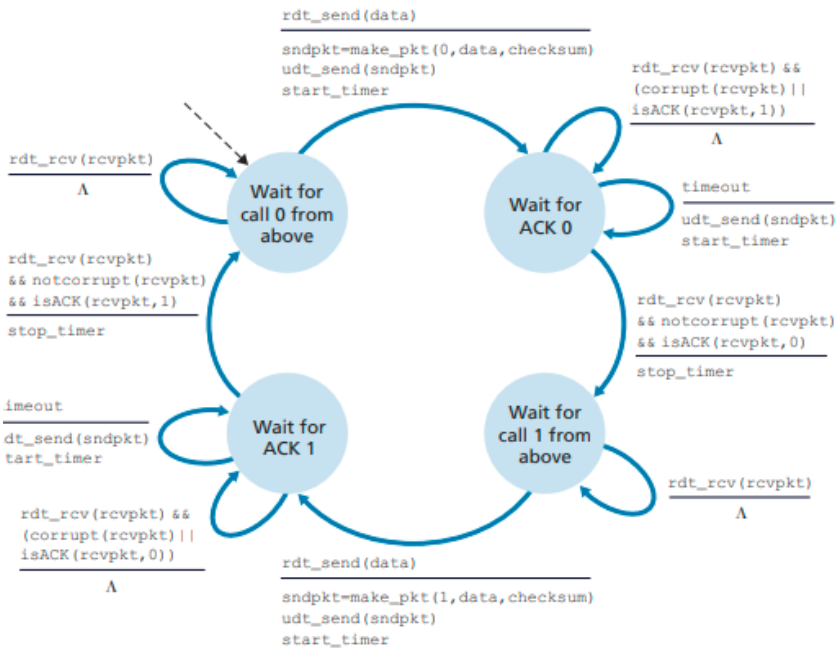
- **RDT 2.0**: Adds error detection and uses ACKs/NAKs. The sender waits for an acknowledgement before sending the next packet (stop-and-wait). Problem: cannot handle corrupted ACK/NAK packets.
- **RDT 2.1**: Adds a 1-bit sequence number so the receiver can detect duplicates caused by retransmissions.
- **RDT 3.0 (what's actually used)**: Drops NAKs entirely. Instead uses a **timeout mechanism**: if no ACK arrives before the timer expires, the sender retransmits.



a. rdt2.0: sending side



b. rdt2.0: receiving side



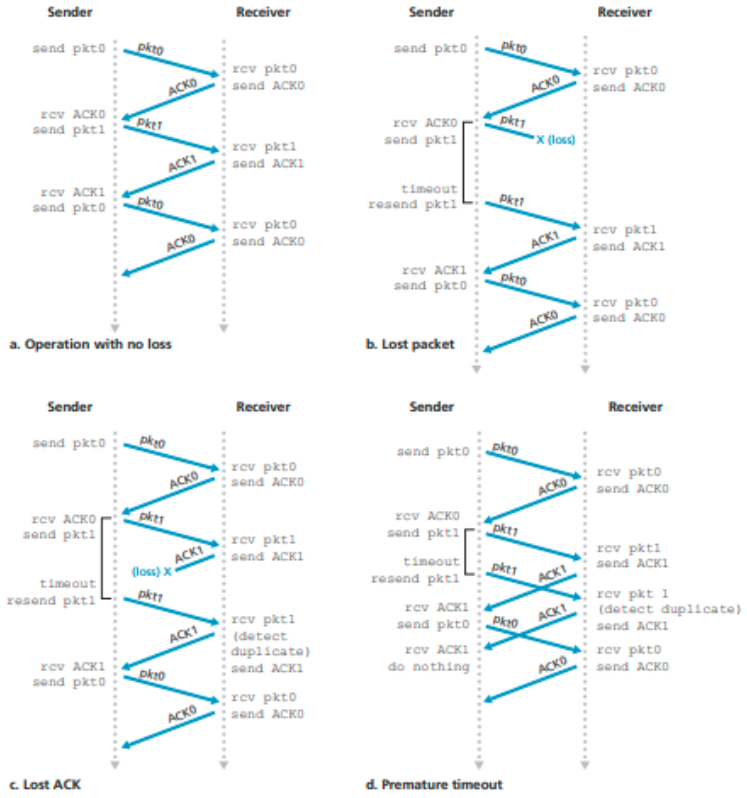


Figure 3.16 ♦ Operation of `rdt3.0`, the alternating-bit protocol

Pipelining: Go-Back-N and Selective Repeat

Stop-and-wait is very inefficient, the channel is idle most of the time. **Pipelining** allows the sender to have multiple in-flight (unACKed) packets.

Go-Back-N (GBN)

- The sender maintains a **window of N** consecutive unACKed packets.
- ACKs are **cumulative**: ACK(n) means all packets up to and including n have been received.
- On receiving ACK(n), the window base moves to n+1.

- A single timer runs for the oldest unACKed packet. On timeout, the sender retransmits **all N packets** in the window.
- The receiver only ACKs the highest **in-order** packet. Out-of-order packets are discarded (or buffered, but not ACKed).

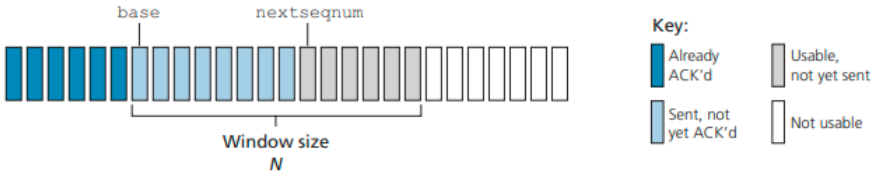
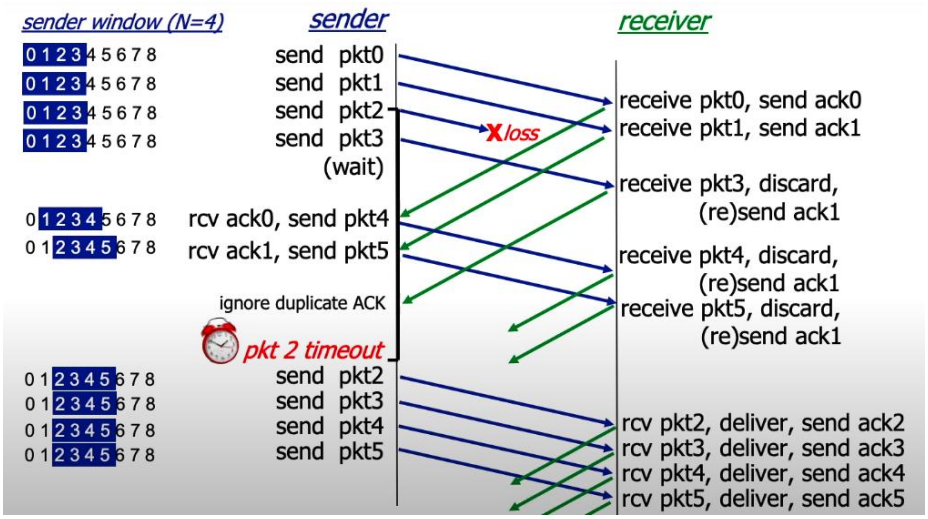


Figure 3.19 ♦ Sender's view of sequence numbers in Go-Back-N



Selective Repeat (SR)

- The receiver ACKs packets **individually** and buffers out-of-order packets.
- The sender has a **separate timer for each unACKed packet**.
- Only the specific timed-out packet is retransmitted.
- Important constraint: the window size must be ≤ half the sequence number space.

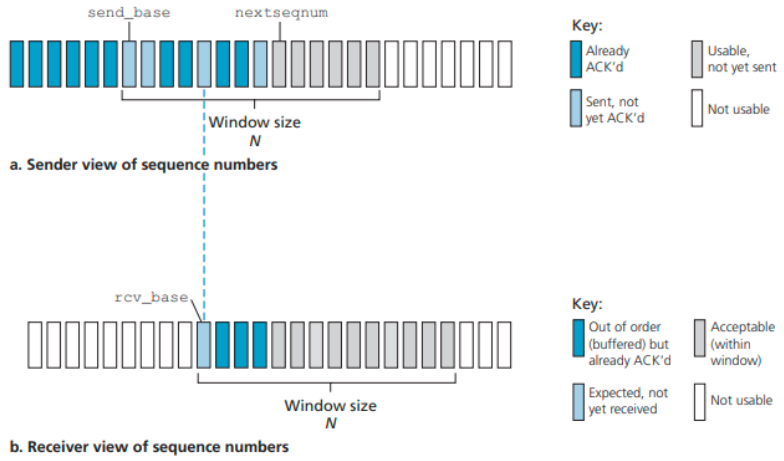
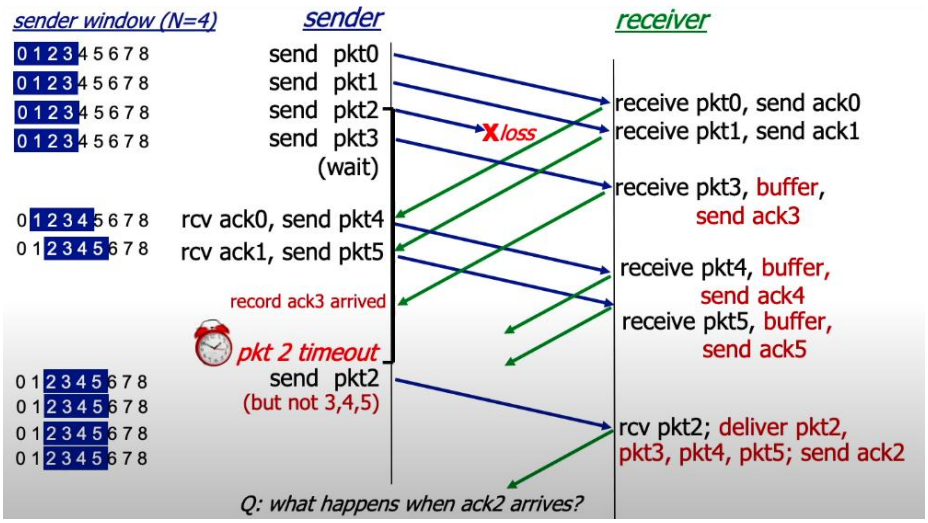


Figure 3.23 ♦ Selective-repeat (SR) sender and receiver views of sequence-number space



3.4: TCP

TCP is a **connection-oriented, reliable, full-duplex** protocol with cumulative ACKs and pipelining. Key properties:

- Point-to-point (one sender, one receiver).
- In-order byte stream.
- Bidirectional, both sides can send and receive at the same time.

Maximum Segment Size (MSS)

The MSS is the maximum amount of **application-layer data** in a single TCP segment (not including headers). It is typically **1460 bytes**, because the link-layer frame is usually 1500 bytes and the TCP+IP headers are 40 bytes. The TCP header alone is normally 20 bytes.

TCP Header

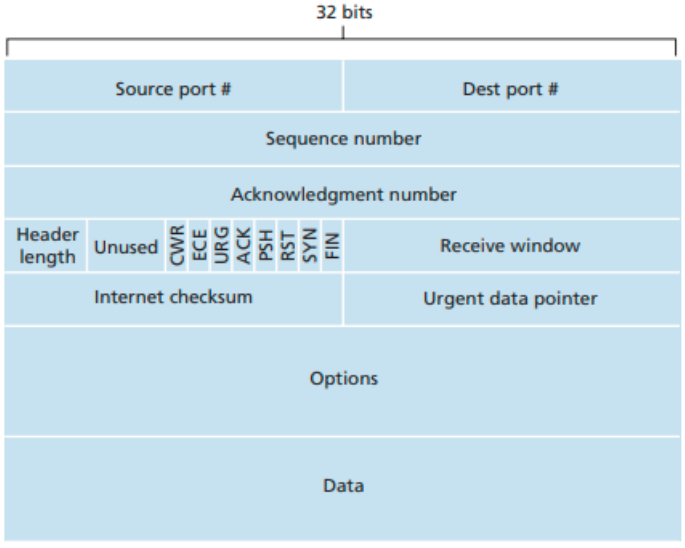


Figure 3.29 ♦ TCP segment structure

Key header fields:

- **Source/Destination port**: For multiplexing/demultiplexing.
- **Sequence number**: The byte-stream number of the first byte in this segment.
- **ACK number**: The next byte the sender expects from the other side.
- **Receive window**: Number of bytes the receiver is willing to accept (used for flow control).

- **Header length**: Length of the TCP header in 32-bit words.
- **Flag field (6 bits)**:
    - **ACK**: Indicates the ACK number field is valid.
    - **RST, SYN, FIN**: Used for connection setup and teardown.
    - **CWR, ECE**: Congestion notification.
    - **PSH, URG**: Rarely used in practice.

Sequence and ACK Numbers in Detail

TCP views data as an **ordered byte stream**. Sequence numbers count bytes, not segments. The ACK number that Host A sends to Host B is the sequence number of the **next byte A expects to receive from B**.

TCP uses **cumulative acknowledgements**: it only ACKs up to the highest in-order byte received. This has an important consequence, if Host A sends three segments and only receives the ACK for the third, it knows all three were received (because Host B would not have ACKed the third without receiving the first two).

Initial sequence numbers are chosen **randomly** to minimize the chance of confusing packets from different connections.

TCP Timeout and Fast Retransmit

- If the timeout expires before an ACK arrives, TCP retransmits and **doubles the timeout** (a limited form of congestion control).
- If the sender receives **3 duplicate ACKs**, it infers packet loss and retransmits immediately, this is called **TCP fast retransmit**.

TCP Flow Control

Flow control prevents the sender from overwhelming the receiver's buffer. The **receive window (rwnd)** field in the TCP header tells the sender how much free buffer space the receiver has:

rwnd = RcvBuffer: (LastByteRcvd: LastByteRead)

The sender keeps the amount of unACKed data below rwnd. If rwnd = 0, the sender continues sending **tiny 1-byte probe segments** to detect when the buffer frees up.

TCP Connection Establishment (3-Way Handshake)

9. **SYN:** The client sends a SYN segment (no app data) with a random initial sequence number. Client enters SYN_SENT state.
10. **SYN-ACK:** The server accepts, allocates buffers, and responds with a SYN-ACK segment containing the server's own random sequence number and ACK = client's SEQ + 1. Server enters SYN_RCVD state.
11. **ACK:** The client allocates buffers and sends an ACK (with ACK = server's SEQ + 1, SYN bit not set). This third segment **may carry application data**. Both sides enter ESTABLISHED state.

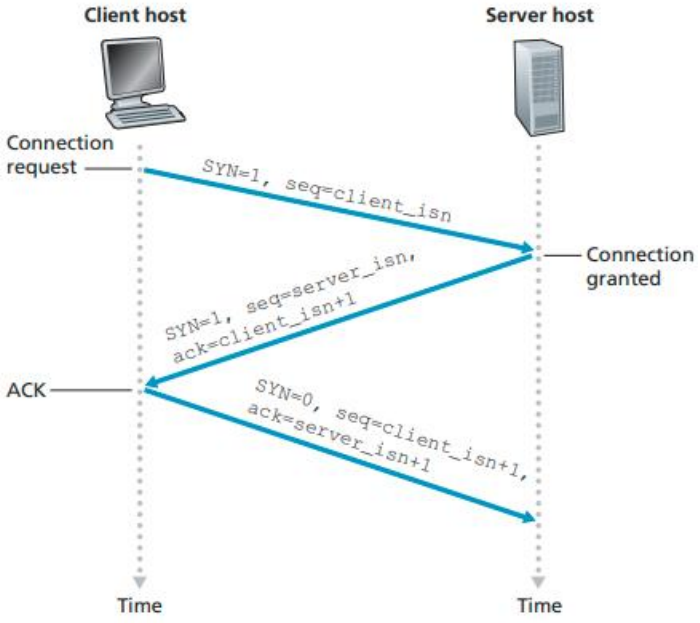


Figure 3.39 ♦ TCP three-way handshake: segment exchange

Closing a connection is a four-way process: the client sends FIN → server responds with FIN-ACK → server sends its own FIN → client responds with FIN-ACK.

3.5: TCP Congestion Control

Congestion control prevents any single TCP connection from overloading the network. Key observations:

- Throughput can never exceed the link capacity.
- Delay increases sharply as utilization approaches capacity.
- Packet loss and retransmission waste bandwidth (duplicate data).
- Upstream capacity is wasted for packets that are dropped downstream.

TCP uses **end-to-end congestion control** (inferred from loss events, not explicit network feedback).

AIMD (Additive Increase, Multiplicative Decrease)

The core idea: **probe for bandwidth** by gradually increasing the sending rate until loss occurs, then cut back.

- Increase the congestion window (cwnd) by **1 MSS every RTT** (additive increase).
- On loss, cut cwnd **in half** (multiplicative decrease).
- AIMD optimizes congested flow rates network-wide and has provably good stability properties.

The **congestion window (cwnd)** constrains unACKed data: ***LastByteSent: LastByteAcked ≤ min{cwnd, rwnd}***. The sender's rate is approximately cwnd/RTT.

TCP Slow Start

When a new connection begins, cwnd starts at **1 MSS** and **doubles every RTT** (exponential growth). This continues until:

- A loss event occurs, or
- cwnd reaches **ssthresh** (slow-start threshold), at which point TCP switches to AIMD.

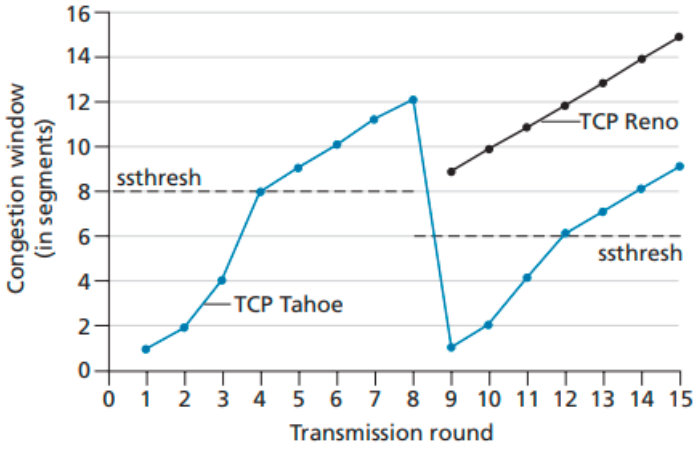
ssthresh is set to half of cwnd at the last loss event.

TCP Tahoe vs. TCP Reno

- **TCP Tahoe**: On any loss event (timeout or 3 duplicate ACKs), set cwnd = 1 MSS and enter slow start. Tahoe = slow start + AIMD + fast retransmit.

- **TCP Reno**: Distinguishes between loss types:
  - On **timeout**: set cwnd = 1, enter slow start (same as Tahoe).
  - On **3 duplicate ACKs (fast retransmit)**: set cwnd = ssthresh (half of cwnd at loss), skip slow start and go directly to AIMD. This is called **fast recovery**.

In short: Reno = Tahoe + fast recovery.

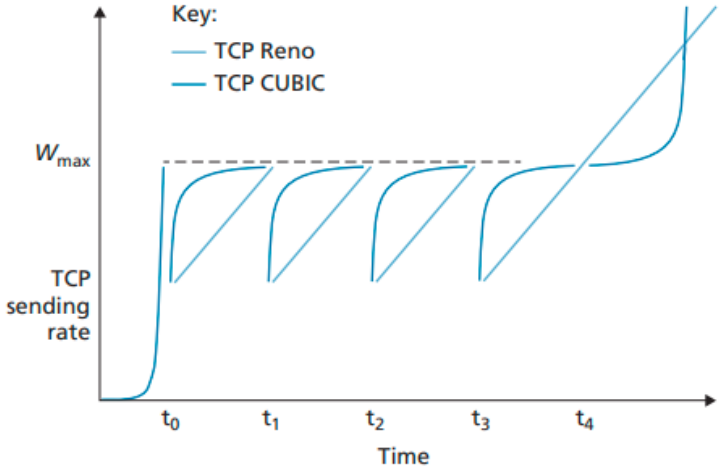


TCP CUBIC

CUBIC is a smarter way to probe for bandwidth than AIMD, and is the **default TCP implementation** on most systems today.

After cutting cwnd in half, CUBIC computes a time **K** at which cwnd will reach the previous maximum window W. It then increases cwnd as a **cubic function of the distance between the current time and K**:

- Far from K → cwnd increases rapidly.
- Close to K → cwnd increases slowly (cautious probing near the known limit).



Average throughput of TCP Reno: approximately **0.75 × W / RTT** (since the sending rate oscillates linearly between W/2 and W).

Delay-Based Congestion Control

An alternative approach: keep the bottleneck link full, but **not fuller**.

- The sender measures RTTs for each packet. The minimum observed RTT (RTT_min) is assumed to be the uncongested RTT.
- If measured RTT ≈ RTT_min → the path is likely uncongested → increase cwnd.
- If measured RTT >> RTT_min → congestion is building → decrease cwnd.

- Advantage: detects congestion **before** packet loss occurs.

TCP Fairness

If N TCP connections share a bottleneck link of capacity R, each should ideally get R/N. TCP's AIMD naturally converges toward fairness: at a loss event, the connection with the larger cwnd suffers a bigger absolute cut, gradually equalizing the shares.

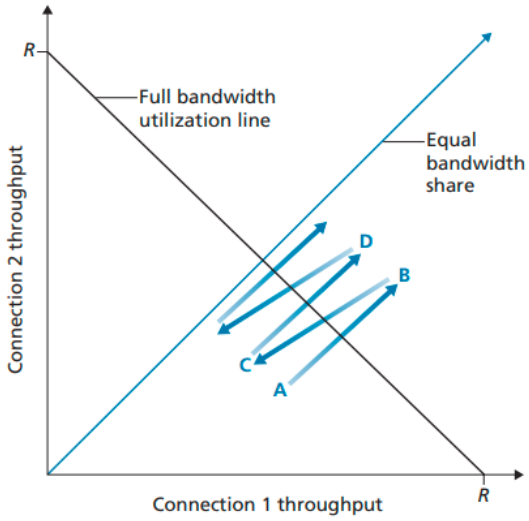


Figure 3.57 ♦ Throughput realized by TCP connections 1 and 2

Notes on fairness:

- UDP traffic ignores congestion control entirely, so it can crowd out TCP.
- A single client with multiple parallel TCP connections gets a disproportionately large share of bandwidth.

3.6: QUIC

QUIC (Quick UDP Internet Connections) is an application-layer protocol designed to improve HTTP, running on top of UDP. Key properties:

- **Connection-oriented**, requires a handshake (like TCP).
- **Reliable and congestion-controlled** (like TCP).
- Transmits data in independent, in-order **streams**, avoiding the HOL-blocking problem of HTTP/1 + TCP.

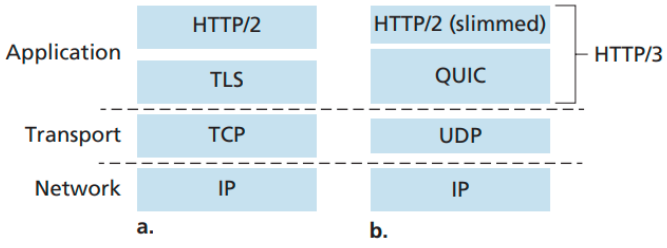


Figure 3.58 ♦ (a) traditional secure HTTP protocol stack, and the (b) secure QUIC-based HTTP/3 protocol stack

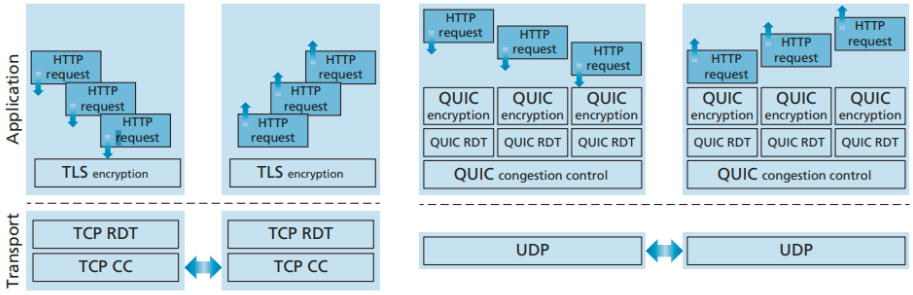


Figure 3.59 ♦ (a) HTTP/1.1: a single-connection client and server using application-level TLS encryption over TCP's reliable data transfer (RDT) and congestion control (CC) (b) HTTP/3: a multi-stream client and server using QUIC's encryption, reliable data transfer and congestion control over UDP's unreliable datagram service

Chapter 4: The Network Layer: Data Plane

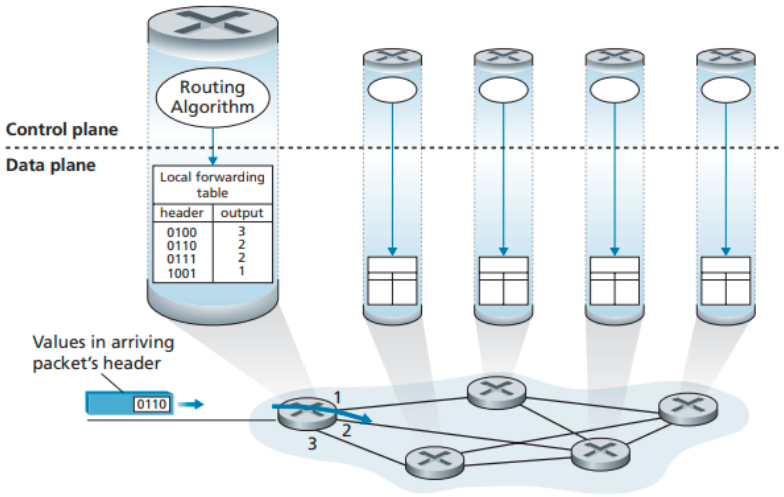
4.1: Overview

The network layer has two distinct functions:

- **Forwarding**: Moving packets from a router's input link to the appropriate output link. This is a **local, per-router** action (the **data plane**).
- **Routing**: Determining the end-to-end path a packet takes from source to destination using routing algorithms. This is a **network-wide** action (the **control plane**).

Two control-plane approaches:

- **Traditional per-router control**: Each router independently runs a routing algorithm to compute its own forwarding table.
- **Software-Defined Networking (SDN)**: A centralized remote controller computes forwarding tables and installs them in routers.



gure 4.2 ♦ Routing algorithms determine values in forward tables

The Best-Effort Service Model

The network layer offers a **best-effort** service: no guarantees on bandwidth, loss, ordering, or timing. This simplicity is a deliberate design choice, combined with sufficient bandwidth provisioning and distributed application-layer infrastructure (e.g., CDNs), it works well enough in practice.

4.2: Inside a Router

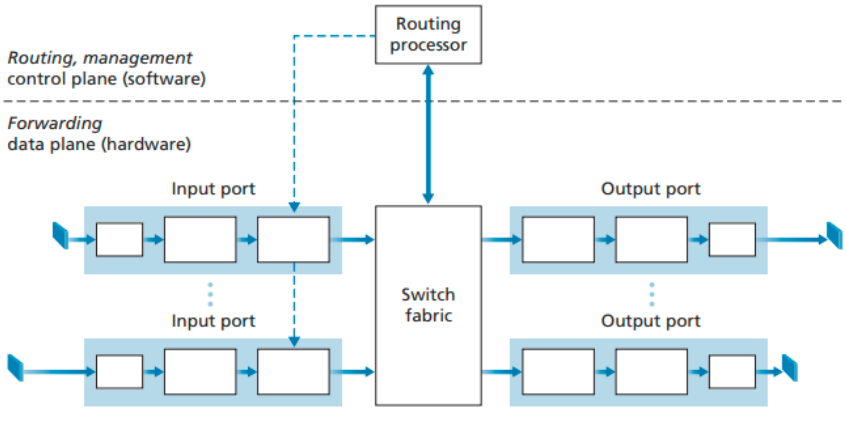


Figure 4.4 ♦ Router architecture

A router has four key components:

- **Input ports**: Perform physical line termination, link-layer processing (e.g., receiving Ethernet frames), and most importantly, the forwarding **lookup**: consulting the forwarding table to determine the correct output port.
- **Switching fabric**: Connects input ports to output ports. The **switching rate** is the throughput of this internal transfer. Ideally, for N input ports each at rate R, the switching rate should be N×R.
- **Output ports**: Buffer packets received from the switching fabric and transmit them onto the outgoing link.
- **Routing processor**: Handles control-plane tasks like maintaining and updating routing tables.

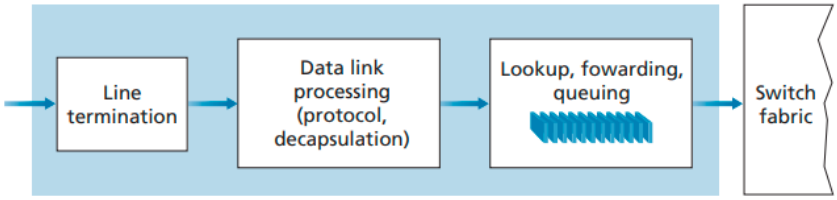


Figure 4.5 ♦ Input port processing

Forwarding Approaches

Forwarding at the input port follows a "match plus action" model:

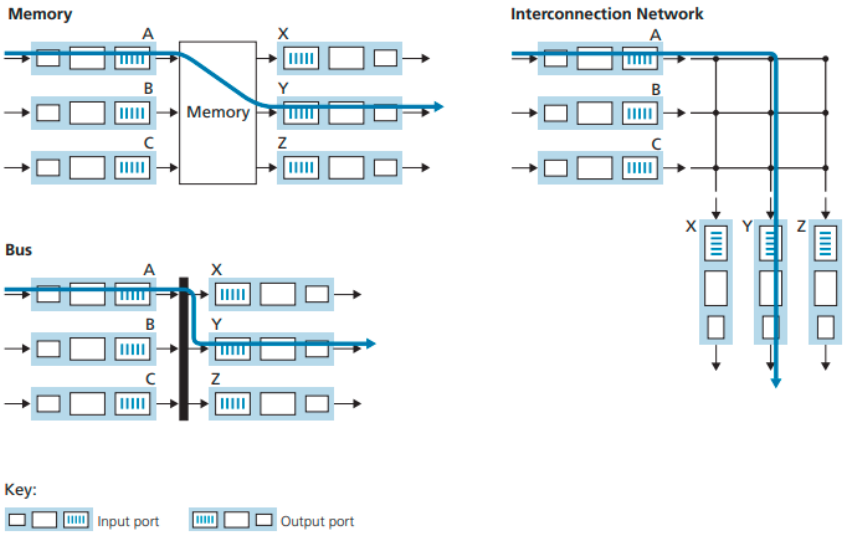
- **Destination-based forwarding**: The output port is determined solely by the packet's destination IP.
- **Generalized forwarding**: The output port can depend on any combination of header fields (e.g., TCP vs. UDP, source IP, etc.).

In destination-based forwarding, the table maps IP ranges to output ports using **longest prefix matching**: the router finds the table entry with the longest address prefix that matches the destination IP.

Prefix	Link Interface
11001000 00010111 00010	0
11001000 00010111 00011000	1
11001000 00010111 00011	2
Otherwise	3

Switching Fabric Technologies

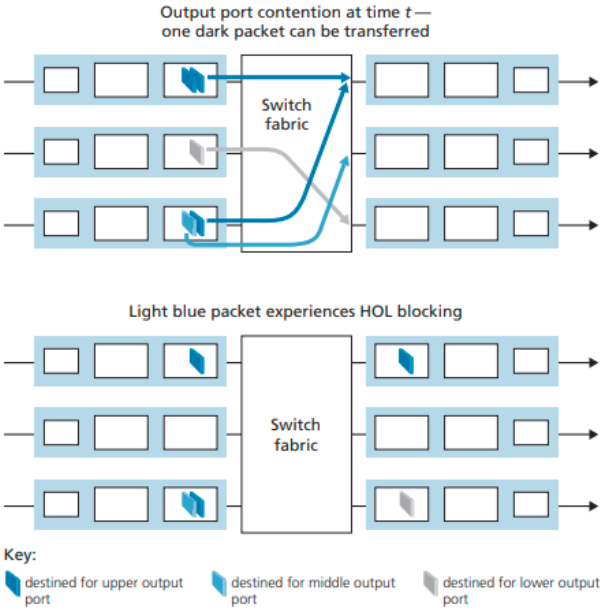
- **Memory-based**: Traditional approach: the CPU copies each packet from input to memory, then to the output. Limited by memory bandwidth.
- **Bus-based**: Packets go directly from input to output over a shared bus. Only one packet can use the bus at a time.
- **Interconnection network (crossbar)**: A multistage switch that connects N inputs to N outputs in parallel. Datagrams are fragmented into fixed-size cells, switched through the fabric in parallel, and reassembled at the output.



Example: Cisco CRS router: Uses 8 switching planes, each with a 3-stage interconnection network, achieving hundreds of Tbps switching capacity.

Input Port Queuing and HOL Blocking

When the switching fabric is slower than ideal, queuing occurs at input ports. **Head-of-line (HOL) blocking** happens when a packet at the front of an input queue blocks packets behind it, even if those packets are destined for free output ports.



gure 4.8 ♦ HOL blocking at and input-queued switch

Output Port Queuing

Buffering is needed when packets arrive from the fabric faster than the output link can transmit. Key decisions:

- **Drop policy**: Which packet to drop when the buffer is full.
- **Scheduling discipline**: Which buffered packet to transmit next.

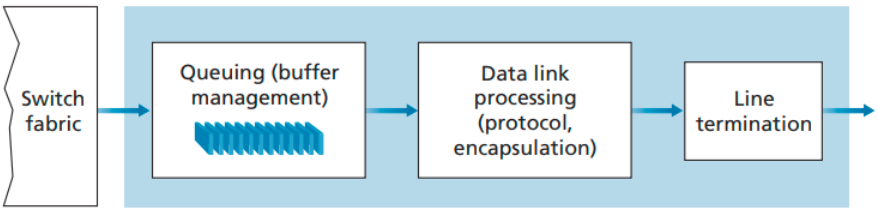


Figure 4.7 ♦ Output port processing

How much buffering? The classic rule of thumb is RTT × C (link capacity). A newer recommendation is RTT × C / √N, where N is the number of flows. Too little buffering causes packet loss; too much causes **bufferbloat**, excessive delays that hurt time-sensitive applications and make TCP slower to detect congestion.

Buffer Management (AQM)

Drop policies are collectively called **Active Queue Management (AQM)** algorithms:

- **Tail drop**: Simply drop newly arriving packets when the buffer is full.
- **Priority drop**: Drop packets based on a priority scheme (e.g., drop data packets before control messages).

Packet Scheduling

- **FIFO**: First come, first served.
- **Priority queuing**: Traffic is classified; packets from the highest-priority non-empty queue are sent first.
- **Round Robin (RR)**: Alternates service between classes in a cycle, rather than strict priority.
- **Weighted Fair Queuing (WFQ)**: A generalized RR where each class gets a weighted share. Class i with weight w_i gets a throughput share of R × w_i / Σw, providing per-class bandwidth guarantees.

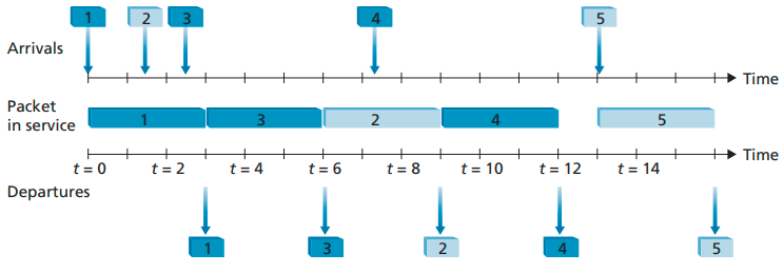


Figure 4.14 ♦ The priority queue in operation

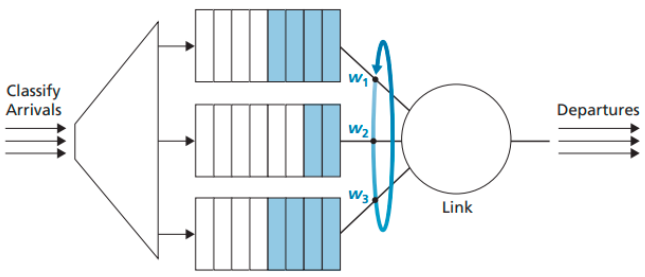
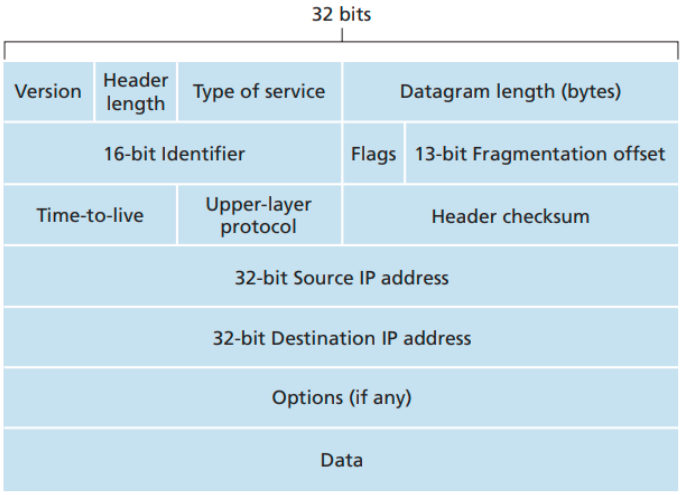


Figure 4.16 ♦ Weighted fair queuing

Network neutrality is the principle that ISPs should not block, throttle, or engage in paid prioritization of traffic.

4.3: The Internet Protocol (IP)

IPv4 Datagram Format



igure 4.17 ♦ IPv4 datagram format

Key header fields:

- **Version**: Distinguishes IPv4 from IPv6.
- **Header length**: Indicates where the payload begins. Usually 20 bytes, but variable due to the options field.
- **Type of Service (TOS)**: Differentiates traffic types (e.g., real-time vs. non-real-time).
- **Datagram length**: Total length of the datagram (header + data) in bytes.
- **Identifier, Flags, Fragmentation offset**: Used for IP fragmentation, where large datagrams are split into smaller ones and reassembled at the destination. Note: **IPv6 does not allow fragmentation at routers** (only at end systems).
- **TTL (Time to Live)**: Decremented by 1 at each router. If it reaches 0, the packet is dropped. Prevents datagrams from circling the network forever.
- **Upper-layer protocol**: Indicates which transport protocol the payload belongs to (e.g., 6 = TCP, 17 = UDP). This binds the network and transport layers, just as port numbers bind transport and application layers.
- **Header checksum**: Computed over all 16-bit header words. Must be recomputed at every router (since TTL changes). **Removed in IPv6** (left to TCP/UDP).
- **Options**: Allow optional processing. The existence of this field is why the header length field is needed. **Removed in IPv6**.

IPv4 Addressing

An IPv4 address is a **32-bit identifier** associated with a link-layer interface (not a host). Hosts typically have one interface, so we speak of a “host’s IP address.” Routers have multiple interfaces and thus multiple IP addresses. Written in dotted-decimal notation (e.g., 223.1.1.1).

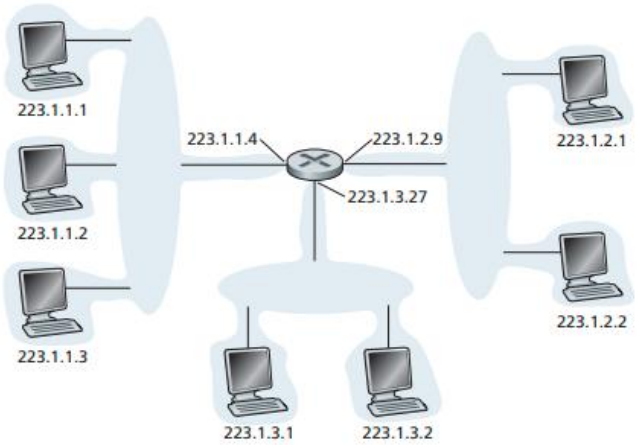


Figure 4.18 ♦ Interface addresses and subnets

Subnets and CIDR

A **subnet** is a group of interfaces that can physically reach each other without passing through a router. IP addresses have two parts:

- **Subnet part** (high-order bits): Shared by all interfaces on the same subnet.
- **Host part** (low-order bits): Unique within the subnet.

The **subnet mask** specifies how many high-order bits define the subnet. Written in **CIDR (Classless Inter-Domain Routing)** notation, e.g., 223.1.1.0/24 means the first 24 bits are the subnet part.

ISP's block:	200.23.16.0/20	11001000 00010111 00010000 00000000
Organization 0	200.23.16.0/23	11001000 00010111 00010000 00000000
Organization 1	200.23.18.0/23	11001000 00010111 00010010 00000000
Organization 2	200.23.20.0/23	11001000 00010111 00010100 00000000
… …		…
Organization 7	200.23.30.0/23	11001000 00010111 00011110 00000000

Organizations receive IP blocks from their ISP, which in turn gets them from ICANN.

DHCP (Dynamic Host Configuration Protocol)

DHCP allows a host to **dynamically obtain an IP address** when joining a network. The DHCP server typically runs on the router. DHCP runs over **UDP** (server port 67, client port 68).

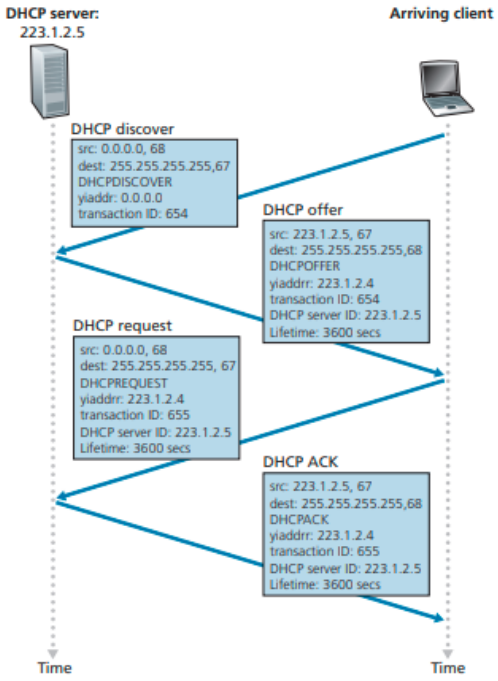


Figure 4.24 ♦ DHCP client-server interaction

The DHCP process:

12. **DHCP Discover**: The client broadcasts a discover message (source IP 0.0.0.0, destination IP 255.255.255.255) with a transaction ID, since all DHCP clients initially share the same IP.
13. **DHCP Offer**: The server responds (also broadcast) with an available IP address, using the same transaction ID.
14. **DHCP Request**: The client requests a specific IP (which may or may not be the offered one). Note: steps 1–2 are optional if the client already knows what address it wants.
15. **DHCP ACK**: The server confirms the assignment, including the address lifetime.

The DHCP server also provides: the first-hop router address, the DNS server's name and IP, and the subnet mask.

NAT (Network Address Translation)

NAT lets all devices on a local network share a **single public IPv4 address**. Internally, devices use private IPs (e.g., from 10.0.0.0/8). The NAT router translates between private and public addresses using port numbers.

How it works:

- **Outgoing datagrams:** The NAT router replaces the (private source IP, port) with (NAT public IP, new port) and records the mapping in its **NAT translation table**.
- **Incoming datagrams:** The router looks up the destination port in its table and replaces (NAT IP, port) with the corresponding (private IP, original port).

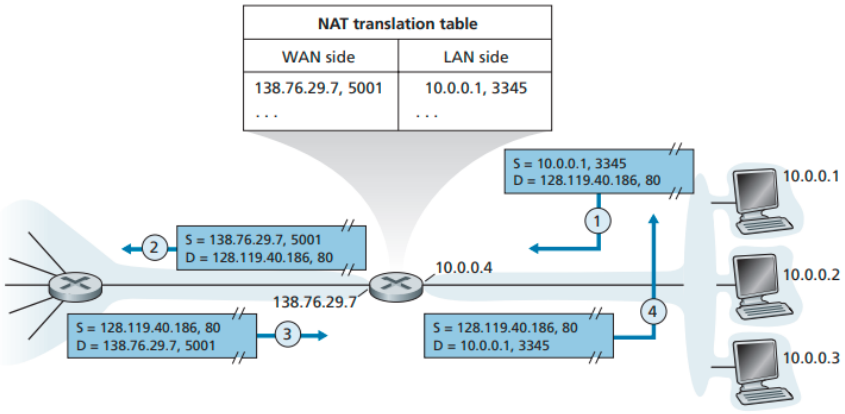


Figure 4.25 ♦ Network address translation

Advantages:

- Conserves the limited IPv4 address space.
- Internal hosts can change addresses without notifying the outside world.
- The ISP can be changed without reconfiguring the internal network.
- Internal devices are not directly addressable from outside (a security benefit).

Disadvantages:

- Routers "should" only process up to layer 3, but NAT inspects port numbers (layer 4).
- Violates the end-to-end principle by modifying port numbers.
- Makes inbound connections awkward (NAT traversal).
- The real address shortage fix is IPv6.

IPv6

Motivations for IPv6: larger address space, faster packet forwarding (fixed-length header, no fragmentation), and support for different treatment of traffic flows.

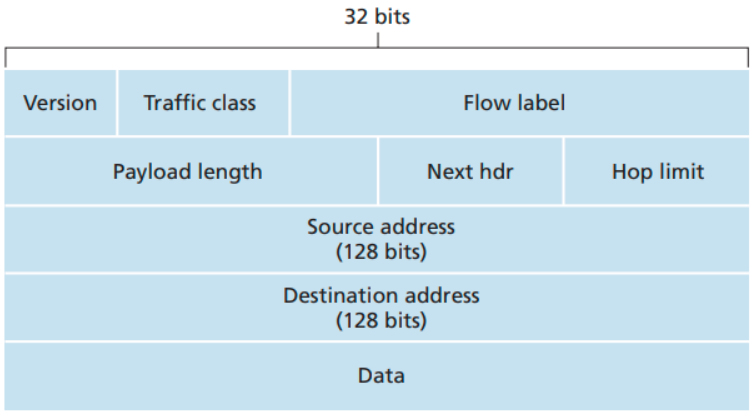


Figure 4.26 ♦ IPv6 datagram format

Key IPv6 header fields:

- **Source/Destination addresses**: Now **128 bits** each.
- **Flow label**: Identifies datagrams belonging to the same "flow" (concept intentionally left loosely defined).
- **Traffic class**: Like TOS in IPv4, used to prioritize traffic.
- **Next header**: Identifies the following protocol (same role as upper-layer protocol in IPv4). Can also point to optional extension headers in the payload.

- **Hop limit**: Equivalent to TTL in IPv4.

Removed from IPv6: Fragmentation (routers drop oversized packets instead), header checksum (delegated to TCP/UDP), and the options field (replaced by next-header chaining).

IPv4-to-IPv6 Transition: Tunneling

To allow IPv6 packets to traverse IPv4 networks, **tunneling** encapsulates an IPv6 datagram inside an IPv4 datagram.

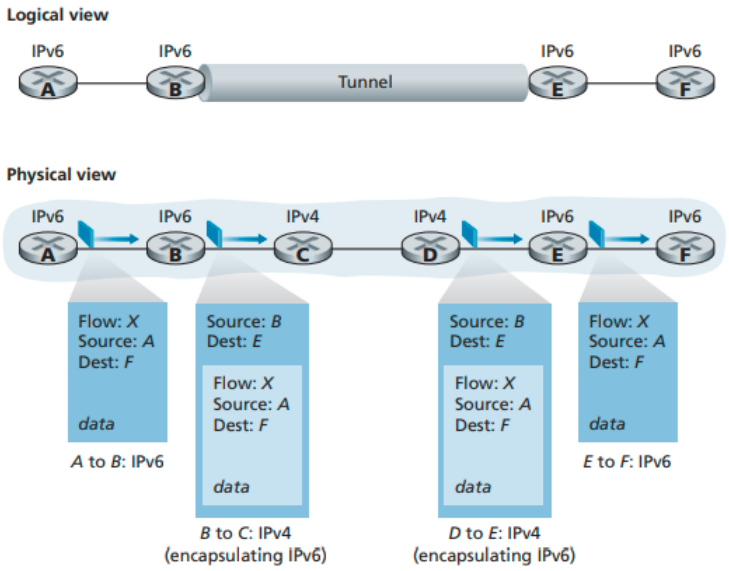


Figure 4.27 ♦ Tunneling

Chapter 5: The Network Layer: Control Plane

5.1: Routing Algorithms

Two categories of routing algorithms:

- **Centralized (link-state)**: Computes least-cost paths using complete, global knowledge of the network topology.
- **Decentralized (distance-vector)**: Computes paths iteratively on a per-router basis, each router only knowing about its direct neighbors.

Note: the underlying shortest-path algorithm is the same in both cases, the difference is in how information is gathered and distributed.

Current routing algorithms are **load-insensitive** (they do not adjust to current congestion levels).

5.2: ICMP (Internet Control Message Protocol)

ICMP is used by hosts and routers to communicate network-level information, primarily error reporting and echo request/reply (ping). It sits architecturally just above IP: ICMP messages are carried inside IP datagrams, but is considered a network-layer protocol.

Common use cases:

- When a router drops a packet because its TTL reached 0, it sends an ICMP “TTL expired” message back to the source.
- **Traceroute** exploits this behavior: it sends UDP datagrams with incrementally increasing TTL values (starting at 1). Each router along the path sends back a TTL-expired ICMP message, revealing its identity. When the destination is finally reached, it sends back a “port unreachable” ICMP message (since traceroute uses a fake port number). Traceroute sends 3 packets per TTL to get multiple timing samples from each hop.

Chapter 6: The Link Layer

6.1: Overview and Services

The link layer is responsible for transferring datagrams from one node to a **physically adjacent** node over a single link. A datagram may pass through multiple different link-layer protocols on its journey (e.g., Wi-Fi on one hop, Ethernet on the next).

Services provided:

- **Framing**: Encapsulates a network-layer datagram into a link-layer frame.
- **Link access (MAC)**: A Medium Access Control protocol governs how frames are transmitted onto the link, especially on shared broadcast channels.
- **Reliable delivery**: Guarantees error-free delivery over the link (similar to TCP's reliable transfer, but at the link level).
- **Error detection and correction**: More sophisticated than the Internet checksum at higher layers.
- **Flow control**: Pacing to avoid overwhelming the receiver.

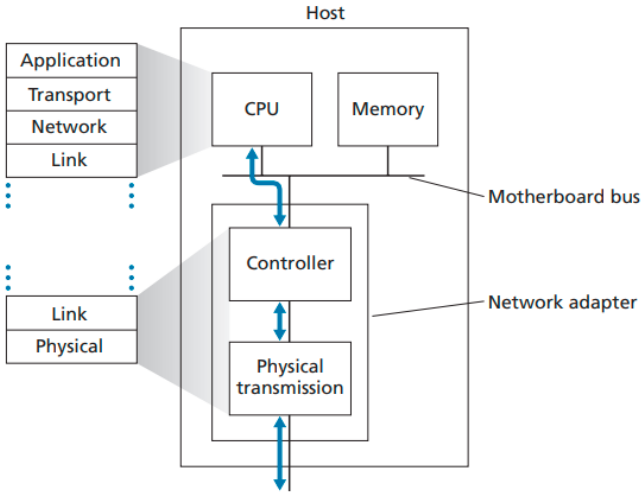


Figure 6.2 ♦ Network adapter: Its relationship to other host components and to protocol stack functionality

Most of the link layer is implemented in hardware, specifically in the **Network Interface Card (NIC)**.

6.2: Error Detection and Correction

Error detection works by appending **error detection and correction bits (EDC)** to each transmitted frame.

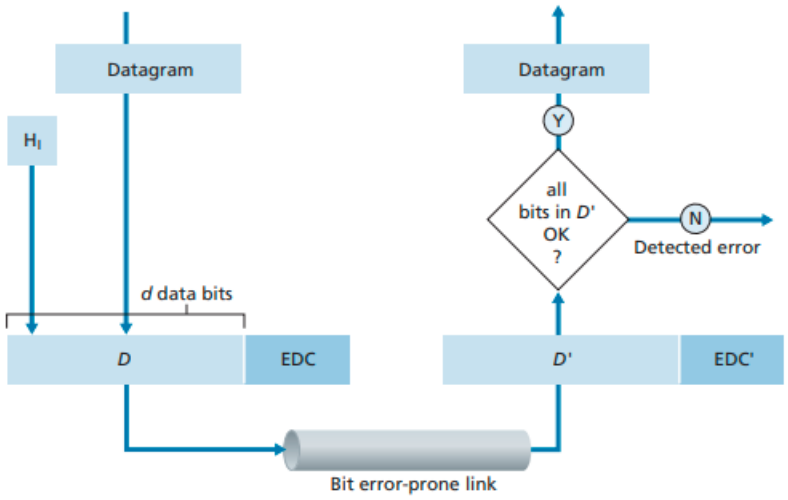


Figure 6.3 ♦ Error-detection and -correction scenario

Parity Checks

- **1D parity**: Append a single bit so the total number of 1s is even. This detects any **odd** number of bit-flip errors.
- **2D parity**: Arrange data in an N×N grid, compute row and column parity bits. This can **detect** 2-bit errors and **correct** 1-bit errors.

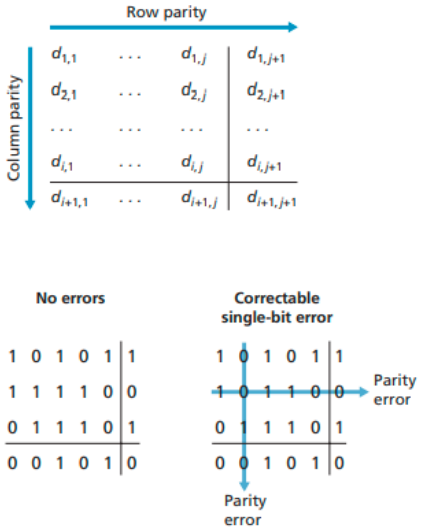


Figure 6.5 ♦ Two-dimensional even parity

CRC (Cyclic Redundancy Check)

CRC is a more powerful error-detection scheme. The key components are:

- **D**: The data (d bits).
- **R**: The CRC redundancy bits (r bits), appended to D.
- **G**: A generator polynomial of r+1 bits, specified by the CRC standard (e.g., CRC-32 IEEE uses a 32-bit generator).

$$G_{\text{CRC-32}} = 100000100110000010001110110110111$$

The sender chooses R such that the concatenation D||R is exactly divisible by G in modulo-2 arithmetic. The receiver divides the received bits by G, if the remainder is non-zero, there's an error.

Key rule: In mod-2 binary arithmetic, addition and subtraction are both XOR, and multiplication/division have no carries or borrows.

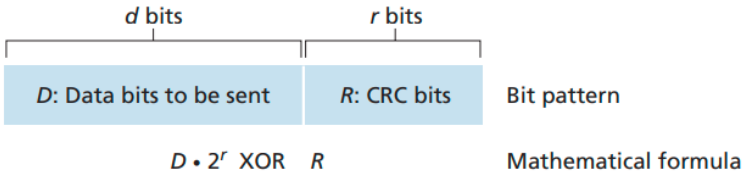


Figure 6.6 ♦ CRC

Computing R:

Multiply D by 2^r (shift left by r bits), then divide by G. The remainder is R:

$$R = remainder(D \times 2^r / G)$$

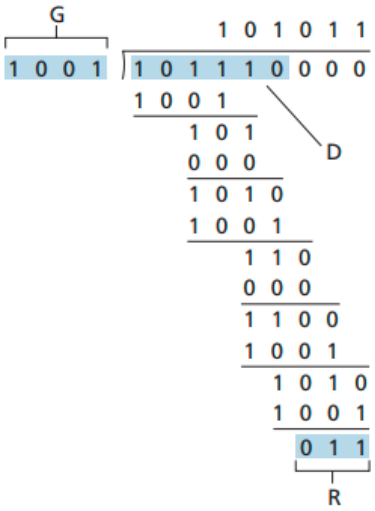


Figure 6.7 ♦ A sample CRC calculation

CRC standards can detect all burst errors of fewer than r+1 bits.

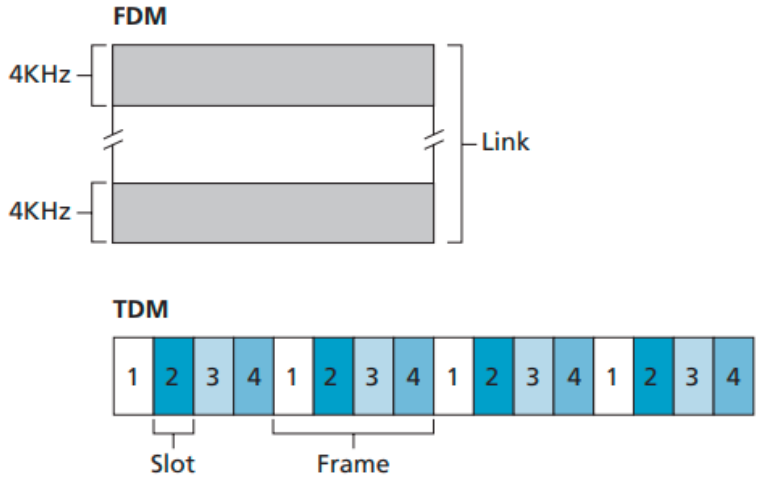
6.3: Multiple Access Protocols

Two types of links: **point-to-point** (e.g., Ethernet switch to host) and **shared medium/broadcast** (e.g., old Ethernet bus, Wi-Fi). A **collision** occurs when a node receives two or more signals simultaneously.

The ideal MAC protocol: distributed, uses only the shared channel for coordination (no out-of-band channel), and allows a single active node to use the full channel rate.

Channel Partitioning Protocols (Collision Avoidance)

- **TDMA (Time Division Multiple Access)**: Time is divided into rounds of fixed-length slots; each station gets one or more dedicated slots. Unused slots go idle.
- **FDMA (Frequency Division Multiple Access)**: The frequency spectrum is divided into bands; each station gets a dedicated band. Unused frequency time goes idle.



- **CDMA (Code Division Multiple Access)**: Each node is assigned a unique code and uses it to encode data. Devices transmit simultaneously; receivers decode the correct signal using the known code. Most commonly used for wireless channels.

Random Access Protocols

In random access, the channel is not divided, collisions are allowed and must be recovered from.

Key properties: Carrier sensing = listen before transmitting; **Collision detection** = abort on collision.

Slotted ALOHA

All frames are the same size. Time is divided into slots (each = 1 frame transmission time). Nodes transmit at slot boundaries. On collision, retransmit in each subsequent slot with probability p.

- Pros: single node gets full rate, decentralized, simple.
- Cons: requires synchronization, wastes slots to collisions and idle time.
- Maximum efficiency: **37%** (at optimal p). **Pure ALOHA** (no slot synchronization) achieves only half of that, about 18%.

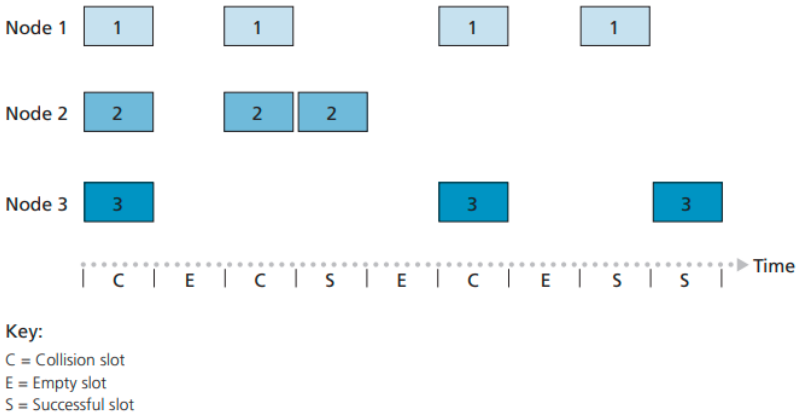


Figure 6.10 ♦ Nodes 1, 2, and 3 collide in the first slot. Node 2 finally succeeds in the fourth slot, node 1 in the eighth slot, and node 3 in the ninth slot

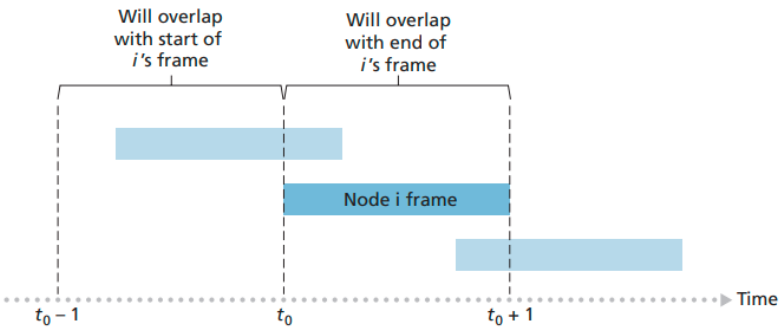


Figure 6.11 ♦ Interfering transmissions in pure ALOHA

CSMA and CSMA/CD

CSMA (Carrier Sense Multiple Access): if the channel is sensed idle, transmit the entire frame. If busy, defer.

CSMA/CD (with Collision Detection): if a collision is detected during transmission, abort immediately to reduce wasted time.

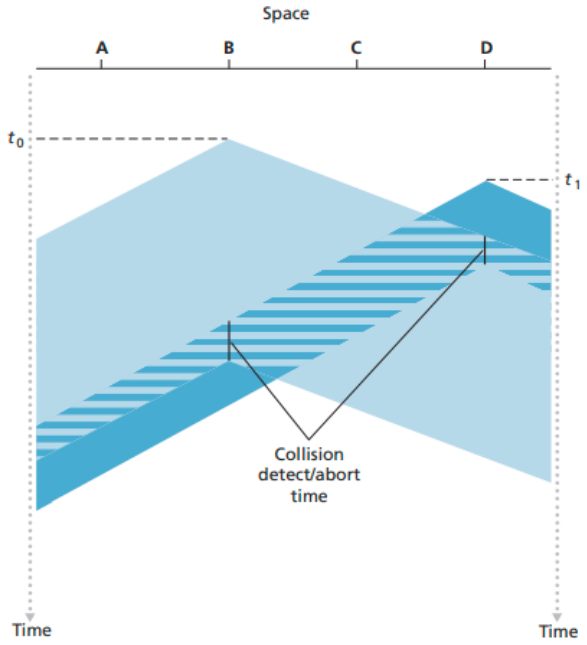


Figure 6.13 ♦ CSMA with collision detection

Ethernet uses CSMA/CD with **binary exponential backoff**: after the n-th collision, choose a random K from {0, 1, ..., 2^m: 1}, wait K × 512 bit times, then try again. More collisions → longer average backoff.

Taking-Turns Protocols

These explicitly allocate channel access (no collisions) while allowing nodes with more data to send for longer.

- **Polling**: A central controller invites nodes to transmit in turn. Pro: good utilization. Con: single point of failure.
- **Token passing**: A token circulates; whoever holds it has the right to transmit.

DOCSIS (Cable Access Networks)

Cable networks combine multiple access methods: **FDM** divides the link into upstream/downstream channels. Within each upstream channel, **TDM** allocates mini-slots, some pre-assigned, some available on demand. Homes request unassigned slots using a **random access** mechanism (CSMA/CD).

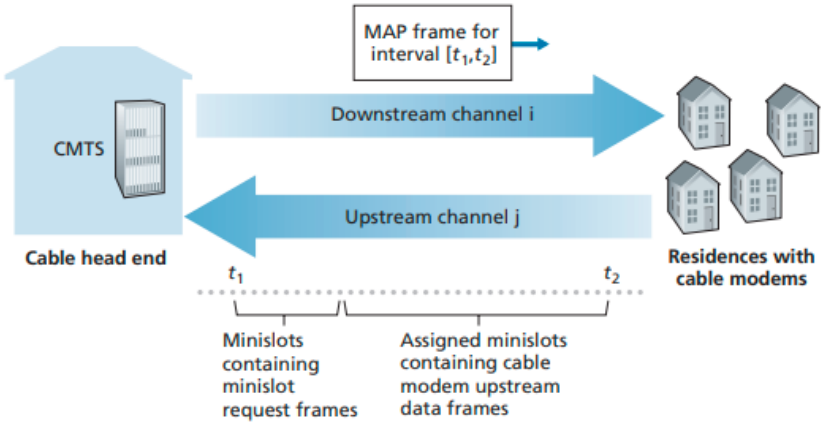


Figure 6.14 ♦ Upstream and downstream channels between CMTS and cable modems

6.4: Switched Local Area Networks

Link-layer switches forward frames based on **MAC addresses** (not IP addresses). Unlike routers, switches operate only at the link layer.

MAC Addresses

A MAC address is a **48-bit identifier** burned into the NIC's ROM (sometimes software-settable). Key facts:

- Each network interface has its own unique MAC address.
- MAC addresses are **flat (non-hierarchical)** and stay the same regardless of network, unlike IP addresses.
- **Switches do not have MAC addresses** because they simply forward frames without processing them.
- An adapter only passes a frame up the protocol stack if the destination MAC matches its own (or is the broadcast address **FF-FF-FF-FF-FF-FF**).

ARP (Address Resolution Protocol)

ARP translates IP addresses to MAC addresses, similar to DNS but only within a subnet. Each interface maintains an ARP table. If a mapping is missing, the adapter broadcasts an ARP request on the subnet; the target responds with a standard (non-broadcast) ARP reply.

When sending a frame to another subnet, the sender must specify the MAC address of the **next hop** (typically the gateway router), not the final destination.

Ethernet Frame Structure

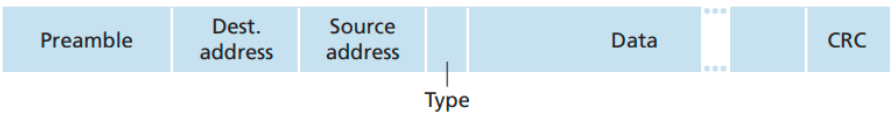


Figure 6.20 ♦ Ethernet frame structure

- **Preamble (8 bytes)**: First 7 bytes are 10101010 (for clock sync); the 8th byte (10101011) signals the frame start.
- **Addresses**: Source and destination MAC.
- **Type**: Indicates the higher-layer protocol (e.g., IP, ARP).
- **Data (46–1500 bytes)**: Payload. The max (1500) is the MTU.
- **CRC**: If the frame fails the CRC check, it is **silently dropped** (no ACK or notification).

Switches

Switches **eliminate collisions** from a network (the only contention is when multiple hosts send to the same destination simultaneously).

Self-learning: Switches build their forwarding table automatically. When a frame arrives, the switch records which interface the source MAC came from. If the destination is unknown, the switch **floods** the frame to all interfaces. This differs from layer 3 (routers), where unknown destinations result in dropped packets, switches can afford to flood because they operate within a local subnet.

Switches vs. Routers

- **Routers** are network-layer devices: they examine network-layer headers (IP) and create a new link-layer frame when forwarding.
- **Switches** are link-layer devices: they forward frames based on MAC addresses without inspecting network-layer headers.

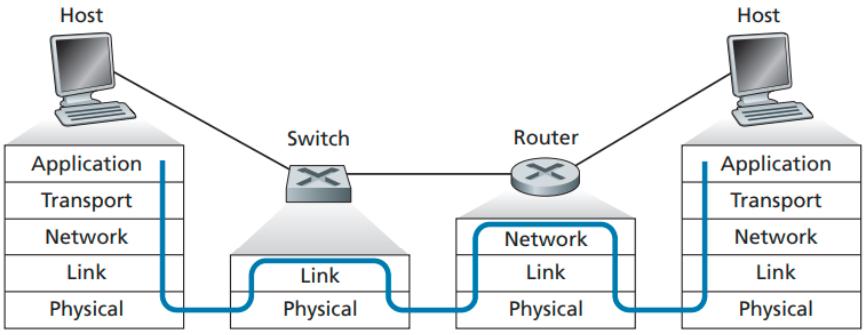


Figure 6.24 ♦ Packet processing in switches, routers, and hosts

6.5: A Day in the Life of a Web Page Request

Here's what happens step by step when a laptop connects to a network and loads a web page:

1. **DHCP:** The laptop broadcasts a DHCP Discover (UDP, dest port 67, src port 68, src IP 0.0.0.0, dest IP 255.255.255.255).
2. The datagram is placed in an Ethernet frame with broadcast MAC (FF:FF:FF:FF:FF:FF).
3. The switch broadcasts the frame. The router's DHCP server receives it and processes the request.
4. The DHCP server sends a DHCP ACK with the laptop's new IP, DNS server address, default gateway address, and subnet mask.
5. The laptop records its configuration. All datagrams with destinations outside the subnet mask go to the default gateway.
6. **DNS:** The laptop creates a DNS query (UDP, port 53) for the website's IP. Before sending, it needs the gateway router's MAC address.
7. **ARP:** The laptop broadcasts an ARP request for the gateway's IP. The router replies with its MAC address.
8. The laptop sends the DNS query, destination IP is the DNS server, but the frame's MAC address is the gateway router's.
9. The DNS query is routed through the network to the DNS server.
10. The DNS server looks up the hostname and replies with the website's IP address.
11. **TCP + HTTP:** The laptop creates a TCP SYN segment (dest port 80, dest IP = website) and sends it via the gateway.
12. The web server responds with a SYN-ACK; the laptop completes the handshake.
13. The laptop sends an HTTP GET request. The server reads the request and sends back an HTTP response with the web page content.
14. The laptop receives the HTTP response and displays the web page.

Chapter 7: Wireless and Mobile Networks

7.1: Wireless Network Taxonomy

Wireless networks can be classified along two axes:

- **Infrastructure-based**: Hosts connect through a base station (the **Access Point, AP**) that links to the wider Internet.
- **Infrastructure-less**: No base station and no connection to a WAN; nodes may relay to reach each other. Sometimes called **MANETs** (Mobile Ad Hoc Networks) or **VANETs** (Vehicular Ad Hoc Networks).

Both types can be **single-hop** (one wireless link to the base station) or **multi-hop** (relaying through several nodes, as in **wireless mesh networks**). Examples: 4G LTE and 802.11 (Wi-Fi) are infrastructure-based, single-hop networks.

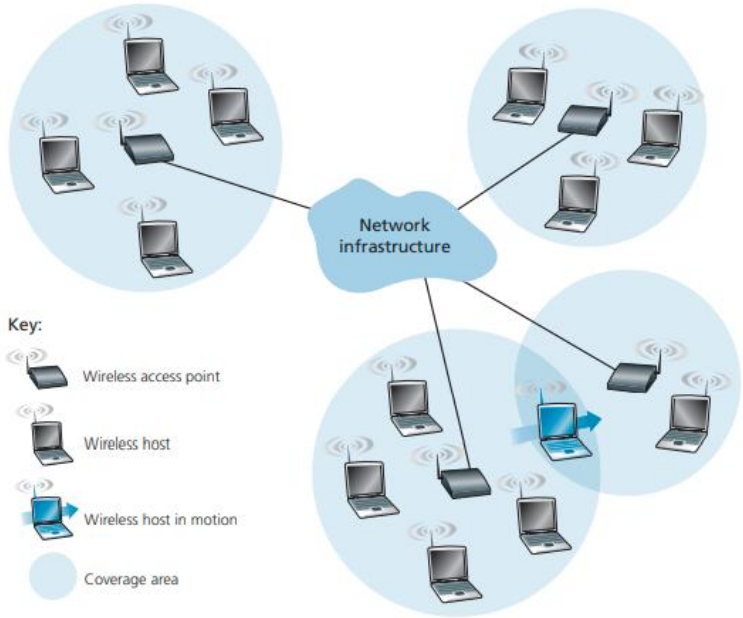


Figure 7.1 ♦ Elements of a wireless network

7.2: Wireless Link Characteristics

Wireless links differ from wired links in three important ways:

- **Path loss**: Signal strength decreases with distance (more significantly at higher frequencies).
- **Interference**: Other devices sharing the same frequency band can cause interference.
- **Multipath propagation**: Reflected signals arrive at slightly different times, distorting the received signal.

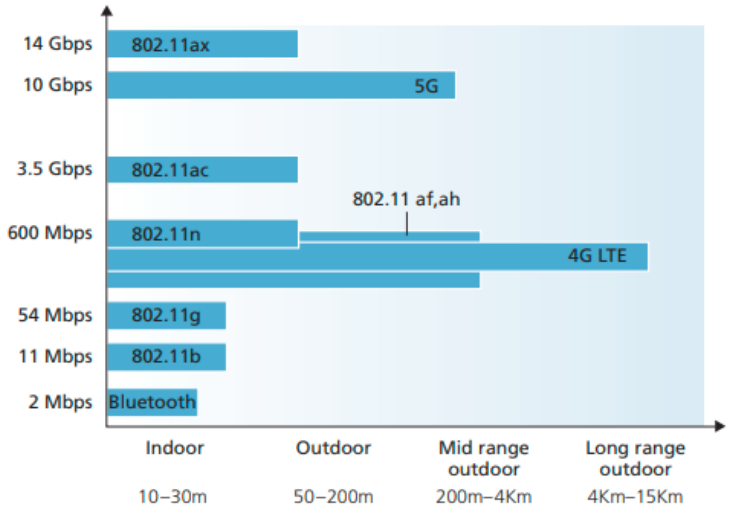
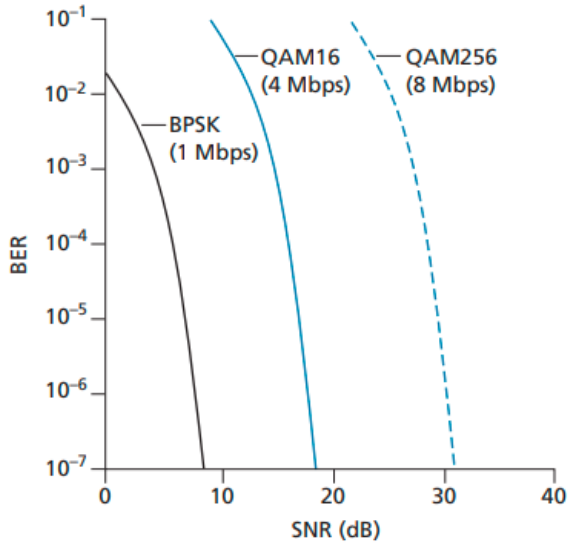


Figure 7.2 ♦ Wireless transmission rates and range for WiFi, cellular 4G/5G and Bluetooth standards (note: axes are not linear)

SNR and BER

For a wireless network to function, it must operate above a minimum **Signal-to-Noise Ratio (SNR)** threshold. Key relationships:

- For a given modulation scheme: **higher SNR → lower Bit Error Rate (BER)**.
- For a given SNR: a modulation scheme with a higher bit rate will have a **higher BER**.
- A sender can increase SNR by increasing transmission power.
- **Dynamic modulation selection** adapts the modulation scheme to current channel conditions.



igure 7.3 ♦ Bit error rate, transmission rate, and SNR

The Hidden Terminal Problem

A **hidden terminal** situation occurs when two nodes (A and C) cannot detect each other's transmissions due to physical obstruction, but their signals overlap at a third node (B). If A and C transmit simultaneously, their signals collide at B, and B cannot decode either transmission.

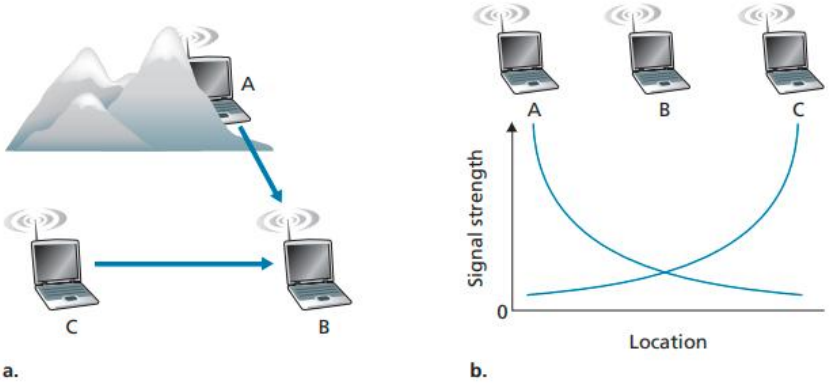


Figure 7.4 ♦ Hidden terminal problem caused by obstacle (a) and fading (b)

7.3: Wi-Fi (802.11)

IEEE 802.11 standard	Year	Max data rate	Range	Frequency
802.11 b	1999	11 Mbps	30 m	2.4 Ghz
802.11 g	2003	54 Mbps	30 m	2.4 Ghz
802.11 n (WiFi 4)	2009	600	70 m	2.4, 5 Ghz
802.11 ac (WiFi 5)	2013	3.47 Gpbs	70 m	5 Ghz
802.11 ax (WiFi 6)	2020 (expected)	14 Gbps	70 m	2.4, 5 Ghz
802.11 af	2014	35–560 Mbps	1 Km	unused TV bands (54–790 MHz)
802.11 ah	2017	347 Mbps	1 Km	900 Mhz

Table 7.1 ♦ Summary of IEEE 802.11 standards

Wi-Fi standards are backwards compatible. All versions use **CSMA/CA** for multiple access and support both base-station and ad-hoc modes.

802.11 Architecture

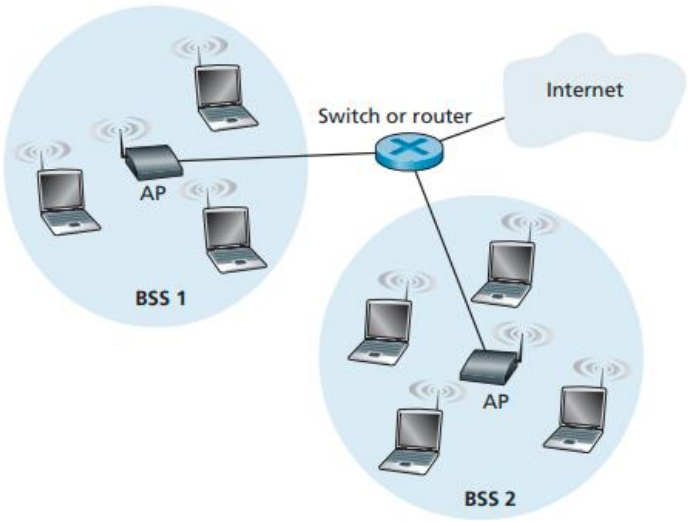


Figure 7.7 ♦ IEEE 802.11 LAN architecture

The **Basic Service Set (BSS)** consists of one or more wireless stations and a central **Access Point (AP)**. Each wireless station has a 6-byte MAC address (just like Ethernet).

To join a network, a host must **associate** with an AP by scanning channels and listening for **beacon frames** (periodically broadcast by the AP). Beacons contain the AP's **SSID** (Service Set Identifier) and MAC address.

802.11 operates in the 2.4–2.4835 GHz range, subdivided into **11 partially overlapping channels**. Two channels are non-overlapping only if separated by at least 4 channels (e.g., channels 1, 6, and 11).

Scanning and Association

- **Passive scanning** (3 steps): The host listens for beacons, sends an association request to the chosen AP, and receives an association response.
- **Active scanning** (4 steps): The host broadcasts a probe request, receives probe responses from nearby APs, then sends an association request and receives a response.

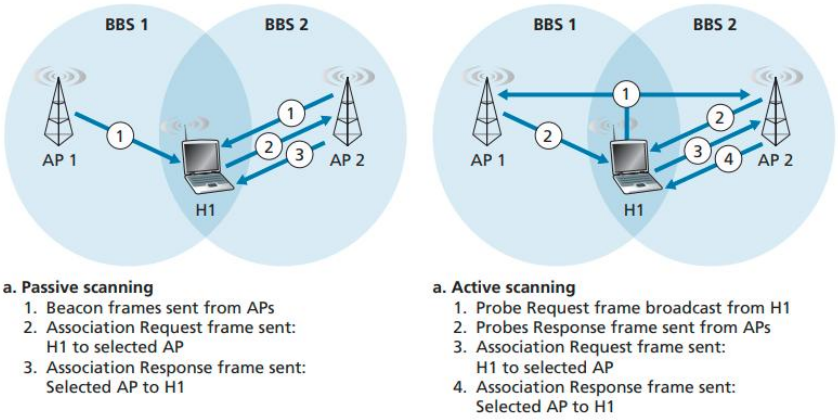


Figure 7.9 ♦ Active and passive scanning for access points

In a **Wi-Fi jungle** (multiple APs in range), the device's firmware typically picks the AP with the strongest signal or fewest connected devices. After associating, the device obtains an IP via **DHCP**.

CSMA/CA (Collision Avoidance)

Why not CSMA/CD (collision detection)? Two reasons:

- A transmitting node's own signal drowns out incoming signals, making it **nearly impossible to sense collisions while transmitting**.
- Hidden terminals mean collisions can happen without the transmitter knowing.

Because wireless has a high bit-error rate, 802.11 uses **link-layer ACKs with retransmission (ARQ)**. After receiving a valid frame (passes CRC), the receiver waits a **Short Inter-Frame Spacing (SIFS)** period, then sends an ACK. If no ACK arrives, the sender retransmits.

CSMA/CA protocol:

16. **Sense the channel.** If idle, wait a **DIFS (Distributed Inter-Frame Spacing)** period, then transmit.
17. If the channel is busy, choose a **random backoff** timer (like CSMA/CD). The timer counts down only while the channel is idle, it freezes when the channel is busy. This prevents two nodes from transmitting at the exact moment the channel becomes free.
18. When the counter reaches zero, transmit the frame and wait for an ACK.
19. If an ACK is received, done. Otherwise, go back to step 2.

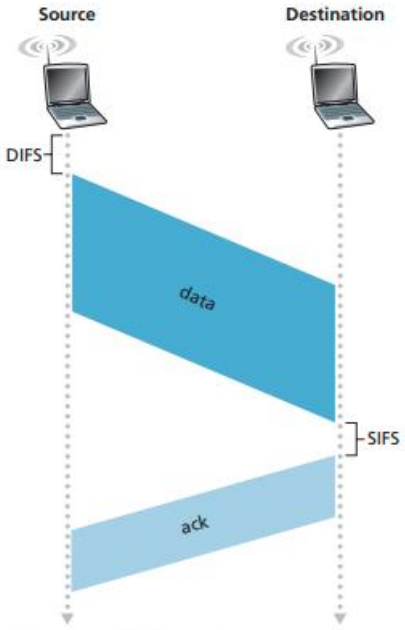


Figure 7.10 ♦ 802.11 uses link-layer acknowledgments

RTS/CTS (Dealing with Hidden Terminals)

802.11 optionally uses **RTS (Request to Send)** and **CTS (Clear to Send)** frames. When a node wants to transmit, it first sends a short RTS to the AP. The AP broadcasts a CTS; any node that receives a CTS not addressed to it defers transmission until it hears the ACK.

Benefits:

- **Mitigates the hidden terminal problem**, all nodes in range of the AP learn the channel is reserved.
- **Minimizes collision time**, collisions can only occur on the short RTS/CTS frames, not on data frames.

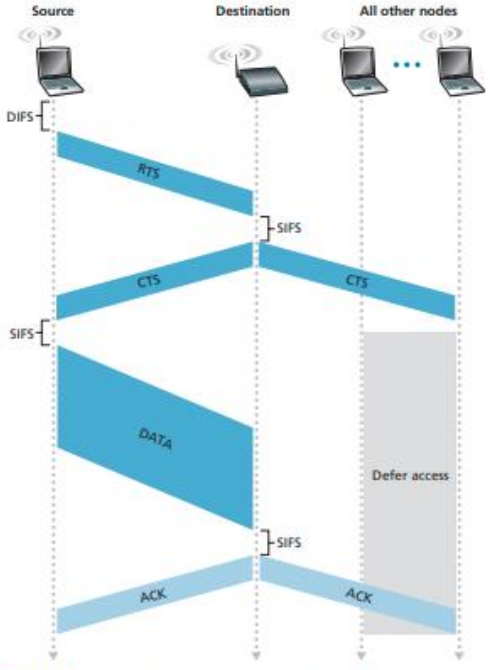


Figure 7.12 ♦ Collision avoidance using the RTS and CTS frames

802.11 Frame Format

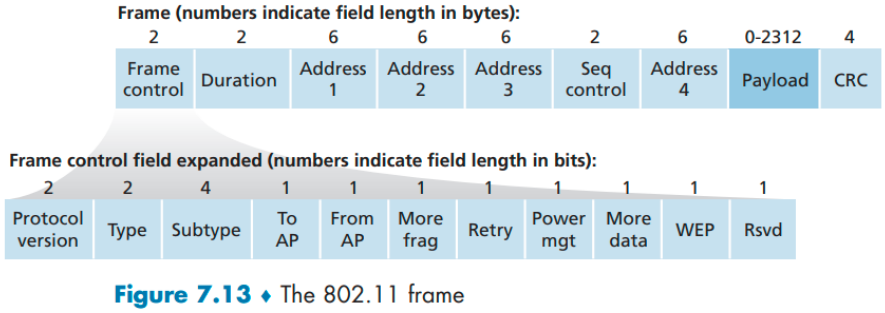
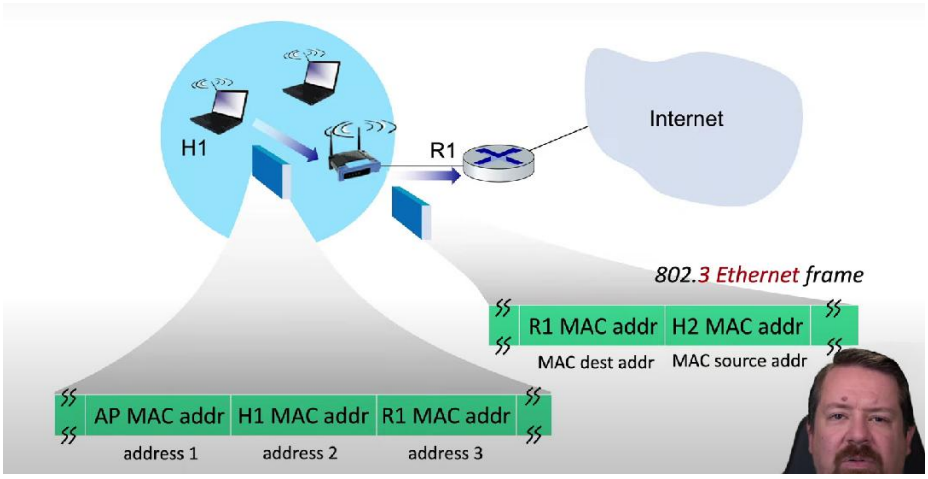


Figure 7.13 ♦ The 802.11 frame

The 802.11 frame has **four address fields**:

15. **Address 1:** MAC of the AP receiving the frame.
16. **Address 2:** MAC of the transmitting host.
17. **Address 3:** MAC of the router interface the BSS's subnet is connected to. This allows the AP to translate between 802.11 frames and Ethernet frames.
18. **Address 4:** Used only in ad-hoc mode.

The **sequence field** serves the same purpose as in TCP (for reliable delivery via ACKs).



Mobility Within and Between Subnets

When a host switches APs **within the same subnet**, the governing switch simply learns the host's new location through ARP. When switching to an AP on a **different subnet**, the host needs a new IP address (via DHCP), and mechanisms exist to prevent active connections from dropping.

Some advanced 802.11 features:

- **Rate adaptation**: Dynamically adjusting the modulation scheme to match channel conditions.
- **Power management**: Minimizing time spent sensing the channel to save battery.

7.4: Cellular Networks: 4G and 5G

Cellular vs. Internet

Similarities: Clear data/control plane separation, edge/core distinction, network of networks, protocol layering.

Differences: Different wireless link layer, mobility as a first-class service, user identity via **SIM card**, and a subscription-based business model.

4G LTE Architecture

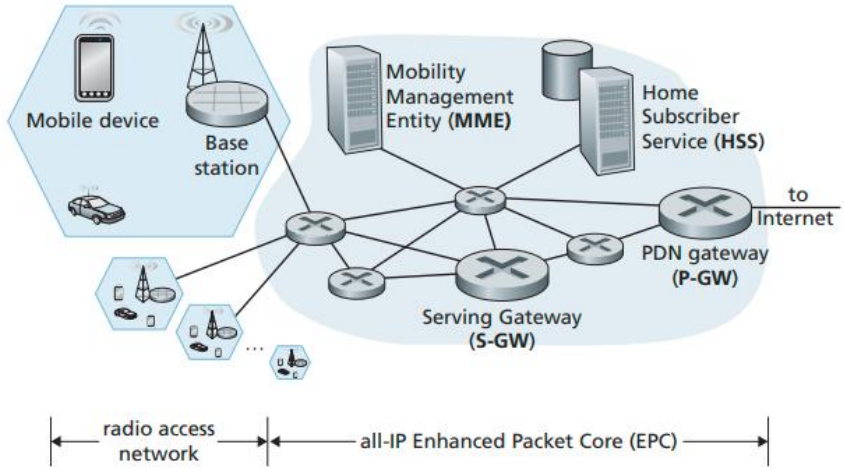


Figure 7.17 ♦ Elements of the 4G LTE architecture

LTE Element	Description	Similar WLAN function(s)
Mobile device (UE: User equipment)	End user's IP-capable wireless/mobile device (e.g., smartphone, tablet, laptop)	Host, end-system
Base Station (eNode-B)	Network side of wireless access link into LTE network	Access point (AP), although the LTE base station performs many functions not found in WLANs
The Mobility Management Entity (MME)	Coordinator for mobile device services: authentication, mobility management	Access point (AP), although the MME performs many functions not found in WLANs
Home Subscriber Server (HSS)	Located in a mobile device's *home* network, providing authentication, access privileges in home and visited networks	No WLAN equivalent
Serving Gateway (S-GW), PDN-Gateway (P-GW)	Routers in a cellular carrier's network, coordinating forwarding to outside of the carrier's network	iBGP and eBGP routers in access ISP network
Radio Access Network	Wireless link between mobile device and a base station	802.11 wireless link between mobile and AP

Table 7.2 ♦ LTE Elements, and similar WLAN (WiFi) functions

Key components:

- **Mobile device (UE: User Equipment)**: Has a globally unique **IMSI (International Mobile Subscriber Identity)** stored on the **SIM card**. The IMSI identifies the subscriber's country and home carrier.
- **Base station (eNode-B)**: Like a Wi-Fi AP, but with additional responsibilities: creating device-specific **IP tunnels** from mobile device to gateways, coordinating with neighboring base stations for handoff, and minimizing inter-cell interference.
- **Home Subscriber Server (HSS)**: A control-plane database storing information about devices belonging to the network. Used with the MME for device authentication.
- **Serving Gateway (S-GW) and PDN Gateway (P-GW)**: Lie in the data path between mobile device and external network (often co-located). The P-GW assigns NAT IP addresses to devices and is the **last LTE element** an outbound datagram passes through. To the outside world, it looks like an ordinary router.
- **Mobility Management Entity (MME)**: A control-plane element (**not on the data plane**) that handles authentication (via HSS), sets up data paths from device to P-GW, and tracks device location across cells.

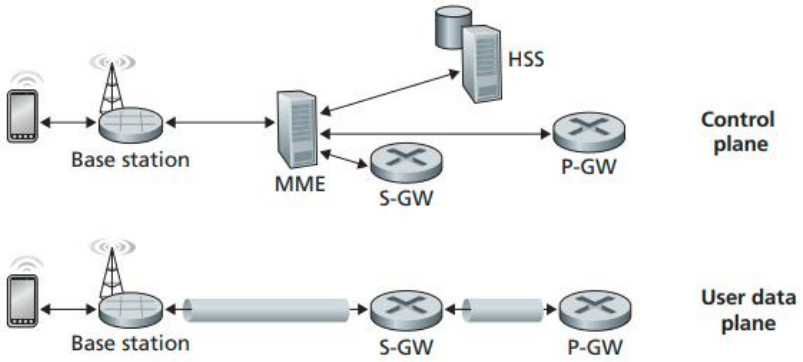


Figure 7.18 ♦ LTE data-plane and control-plane elements

LTE Link-Layer Protocol Stack

4G LTE is an all-IP architecture, the network layer and above work identically to wired networks. The link layer is subdivided into three sublayers:

- **Packet Data Convergence** (uppermost): Performs IP datagram **compression** and **encryption** using keys established during signaling.
- **Radio Link Control (RLC)**: **Fragments** and **reassembles** IP datagrams too large for link-layer frames. Implements **reliable data transfer** via an ACK/NAK-based ARQ protocol.
- **Medium Access Control (MAC)**: Handles transmission scheduling, slot allocation requests, and error detection/correction (CRC).

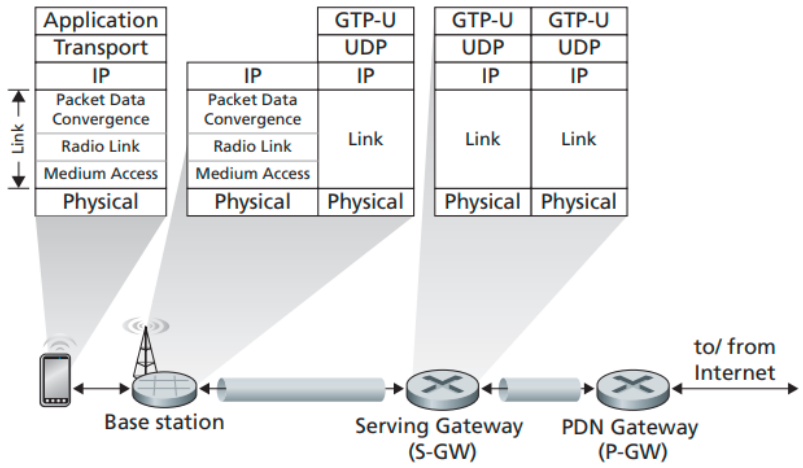


Figure 7.21 ♦ LTE data-plane protocol stacks

Each tunnel between endpoints has a unique **Tunnel Endpoint Identifier (TEID)**. The base station encapsulates datagrams from the mobile device using **GPRS Tunneling Protocol (GTP)** and sends them in UDP segments to the serving gateway. In the reverse direction, it decapsulates incoming tunneled datagrams and sends the extracted IP packet over the wireless hop.

LTE Radio Access

LTE uses **OFDM (Orthogonal Frequency Division Multiplexing)**, a combination of TDM and FDM on the downstream channel. Each active device is allocated one or more **0.5 ms timeslots in one or more frequency channels**. Allocating more slots gives a user a higher data rate.

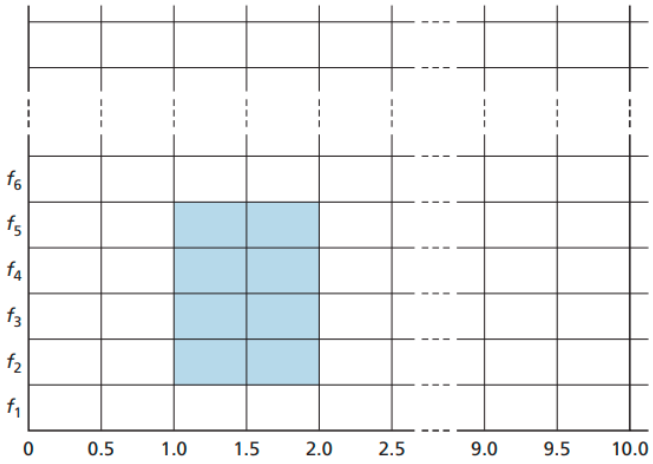


Figure 7.22 ♦ Twenty 0.5-ms slots organized into 10 ms frames at each frequency. An eight-slot allocation is shown shaded.

LTE Network Attachment

Attaching a mobile device to the network involves three phases:

1. **Base station attachment:** The device scans all channels for **primary synchronization signals** (broadcast every 5 ms), then finds the **secondary synchronization signal** for details like channel bandwidth and carrier info. It then selects a base station.
2. **Mutual authentication:** The network identifies the device's IMSI, and the device authenticates the connection (covered in Chapter 8).
3. **Data path configuration:** The MME contacts the P-GW and S-GW to assign a NAT IP address and establish IP tunnels.

Power Management

- **Discontinuous reception**: The device schedules periodic check-ins with the base station for buffered incoming data.
- **Idle state**: After longer periods of inactivity (5–10 seconds), the device enters a deeper sleep.

5G Cellular Networks

5G is **not backwards-compatible** with 4G at the physical layer, requiring new infrastructure.

Key differences from 4G:

- **Millimeter-wave frequencies**: Much faster data rates, but shorter range, easily blocked by obstacles, and susceptible to atmospheric interference.
- **MIMO and beamforming**: Instead of increasing transmission power, 5G base stations use multiple antennas to focus signals directionally. This requires **more and smaller cell stations**.
- **User-Plane Function (UPF)**: Complete control/user-plane separation, allowing packet processing to be pushed to network edges.
- The 4G MME is split into two entities: **AMF (Access and Mobility Function)** and **SMF (Session Management Function)**.

Chapter 8: Security

8.1: What Is Network Security?

Four desirable properties of secure communication:

- **Confidentiality**: Only the sender and receiver should understand the message contents (achieved through **encryption**).
- **Message integrity**: Message contents should not be altered in transit.
- **End-point authentication**: The identities of the communicating parties should be verifiable.
- **Operational security**: Internal networks should be protected from external threats (e.g., via a **firewall**).

An intruder on the channel can potentially: **eavesdrop** (sniff/record messages), **modify**, **insert**, or **delete** messages.

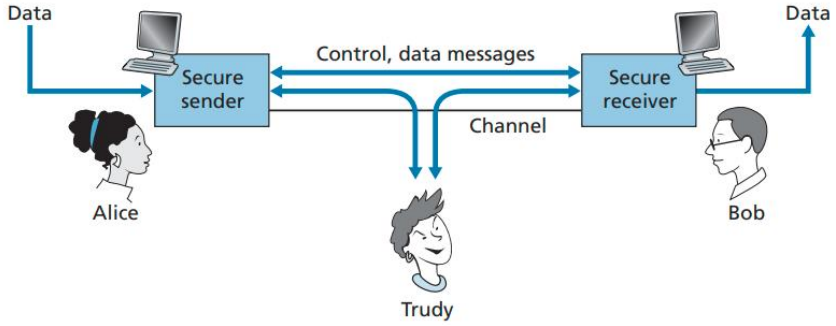


Figure 8.1 ♦ Sender, receiver, and intruder (Alice, Bob, and Trudy)

8.2: Encryption

An encryption algorithm transforms plaintext into ciphertext using a **key**. Two main paradigms:

- **Symmetric key**: Both parties share the same secret key.
- **Public key**: Each party has a key pair: a **public key K^+** (shared openly) and a **private key K^-** (kept secret). The fundamental property: $K^+(K^-(m)) = m$.

Classical Ciphers

- **Caesar cipher**: Shifts each letter by a fixed value k.
- **Monoalphabetic cipher**: Uses an arbitrary letter substitution map.
- **Polyalphabetic cipher**: Like Caesar, but the shift value k changes cyclically, so the same letter may map to different ciphertext characters depending on position.

Attack Scenarios

- **Ciphertext-only**: The attacker has only the encrypted message.
- **Known-plaintext**: The attacker knows some plaintext–ciphertext pairs.
- **Chosen-plaintext**: The attacker can choose a plaintext and obtain its ciphertext.

Block Ciphers

Modern symmetric encryption uses **block ciphers** (used in PGP, TLS, IPSec). The message is divided into blocks of k bits, and each block is mapped through a one-to-one function to produce a k-bit ciphertext block. Typical block sizes: 64 or 128 bits.

Since a full lookup table of size 2^{64} is infeasible, block ciphers simulate random permutations by breaking blocks into smaller chunks and processing them through multiple rounds:

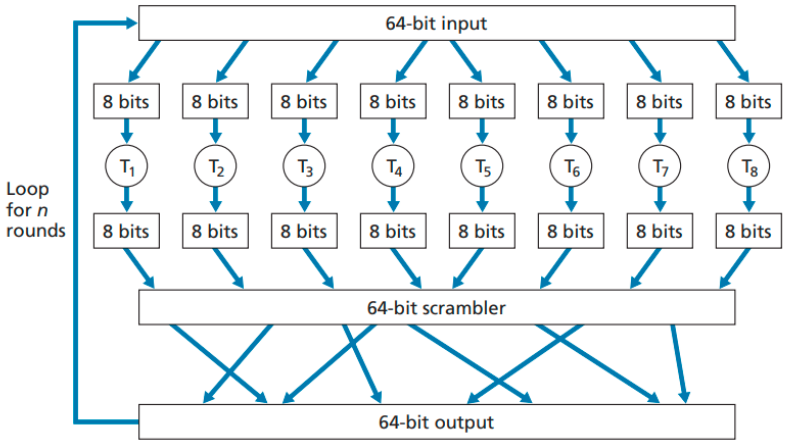


Figure 8.5 ♦ An example of a block cipher

Popular block ciphers: **DES** (64-bit blocks), **3DES**, and **AES** (128-bit blocks).

Cipher Block Chaining (CBC) introduces randomness so that identical plaintext blocks produce different ciphertext, preventing pattern analysis.

Public Key Encryption

Public key cryptography (e.g., Diffie-Hellman, RSA) allows secure communication **without a pre-shared secret**. Alice encrypts with Bob's public key; only Bob can decrypt with his private key.

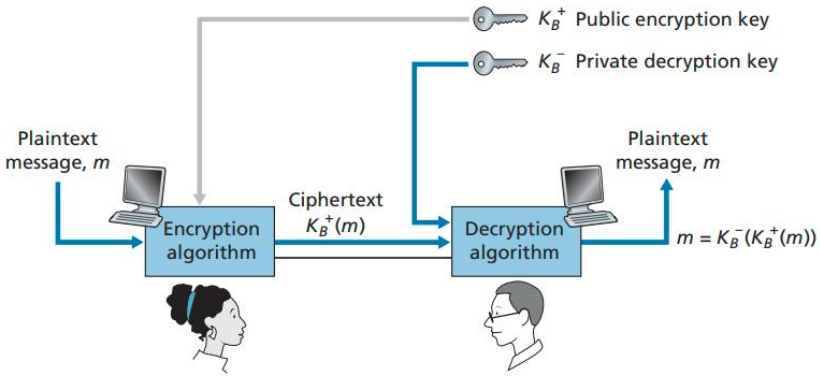


Figure 8.6 ♦ Public key cryptography

8.3: Message Integrity and Authentication

Cryptographic Hash Functions

A **cryptographic hash function H** has the property that it is computationally infeasible to find two inputs x and y where H(x) = H(y). This means:

- You cannot reverse-engineer x from H(x) (no feasible inverse).
- If H(x) = H(y), you can be confident that x = y.

In communication: the sender sends (m, H(m)). If the message is altered to m' in transit, the receiver computes H(m'), finds it doesn't match H(m), and knows the message was tampered with. Common hash functions: **MD5** and **SHA-1** (SHA-1 is more secure due to its longer output).

Message Authentication Code (MAC)

Problem: an attacker could replace (m, H(m)) with (m', H(m')). Solution: incorporate a **shared secret s** into the hash. The sender sends (m, H(m + s)). The receiver, who also knows s, computes H(m + s) and compares. No encryption is needed for integrity, just the shared secret.

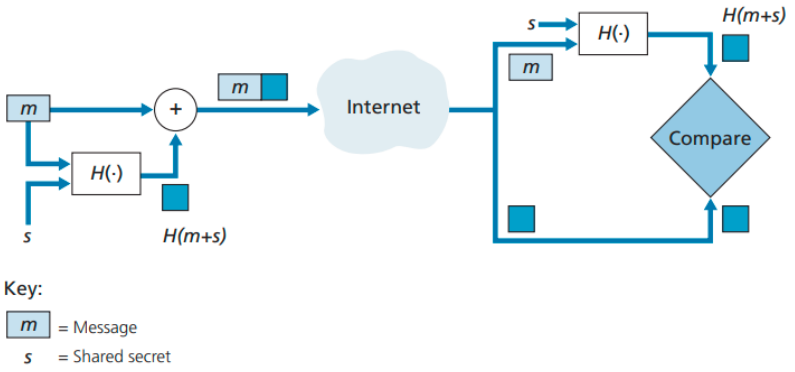


Figure 8.9 ♦ Message authentication code (MAC)

HMAC is the most widely used MAC standard, running with MD5 or SHA-1.

Digital Signatures

A **digital signature** is a verifiable, non-forgeable way to confirm identity. The sender sends the tuple:

$$(m, K^-(H(m)), K^+)$$

The receiver extracts H(m) by computing $K^+(K^-(H(m)))$, then verifies message integrity with standard MAC. The signature works because only the sender knows K^-. The hash (rather than the full message) is signed because it's computationally cheaper.

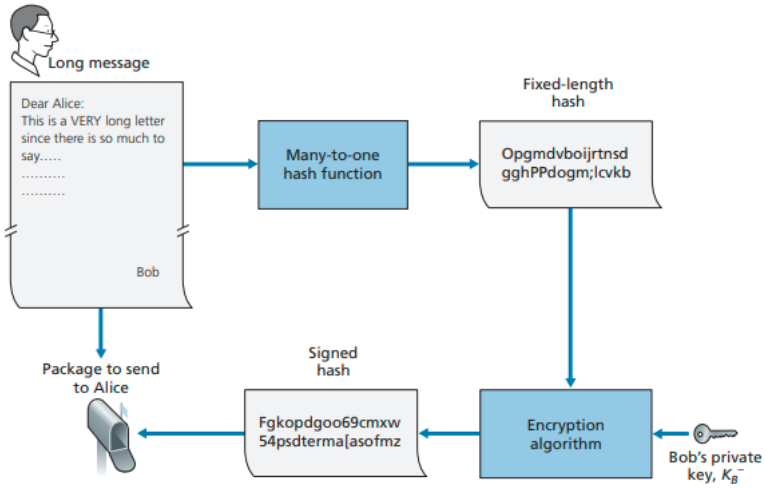


Figure 8.11 ♦ Sending a digitally signed message

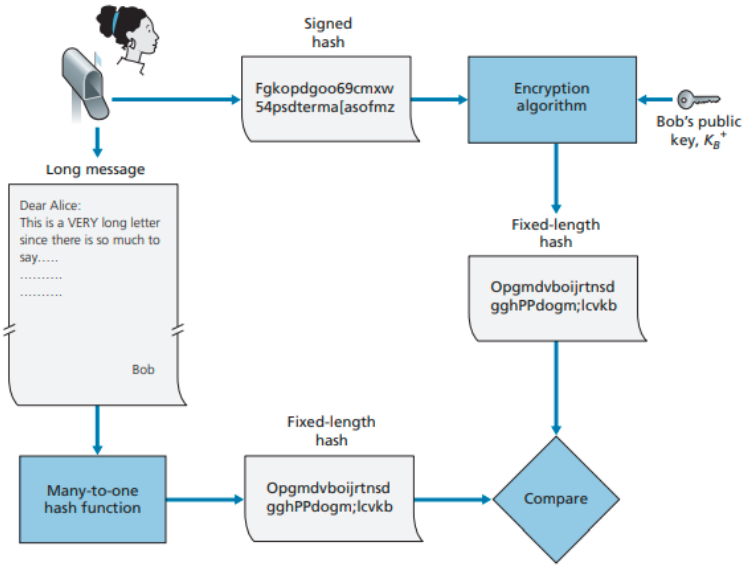


Figure 8.12 ♦ Verifying a signed message

Public Key Certification

How do you know a public key really belongs to who it claims? A **Certification Authority (CA)** issues **certificates** that bind a public key to a specific entity, digitally signed by the CA.

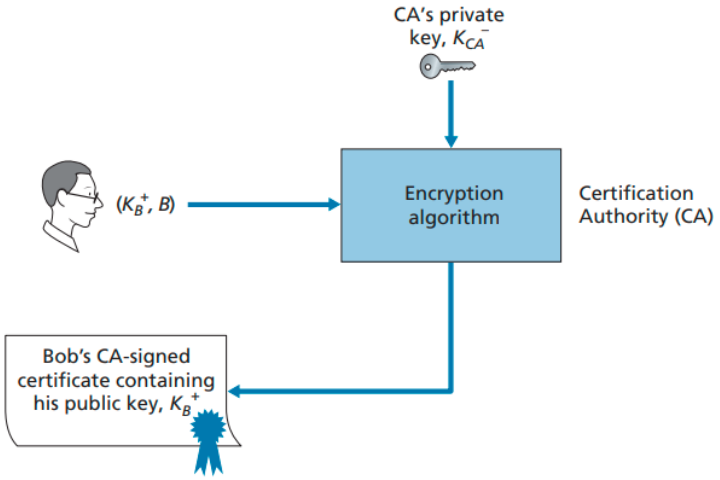


Figure 8.14 ♦ Bob has his public key certified by the CA

This prevents man-in-the-middle attacks where an attacker substitutes their own public key:

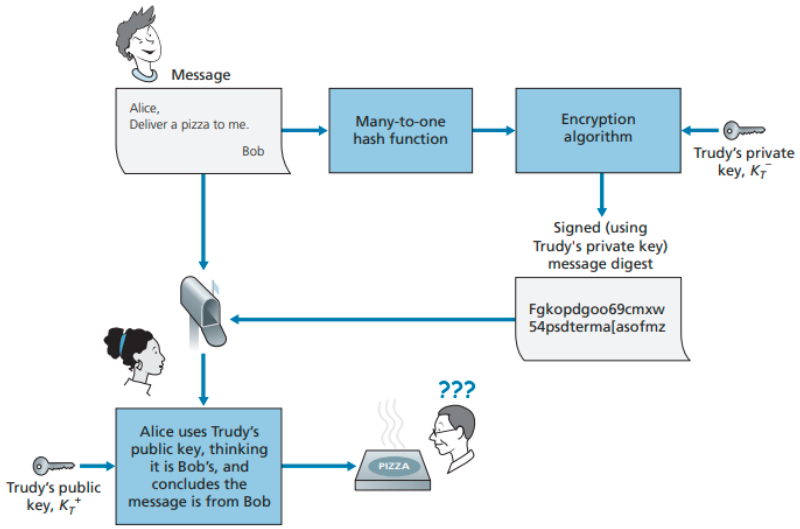


Figure 8.13 ♦ Trudy masquerades as Bob using public key cryptography

8.4: Securing Communication

Secure Email (Session Keys)

Public key encryption is computationally expensive. In practice, communication uses a **session key**: a random symmetric key generated for one session. The session key is securely exchanged using public key cryptography, and then all further communication uses faster symmetric encryption.

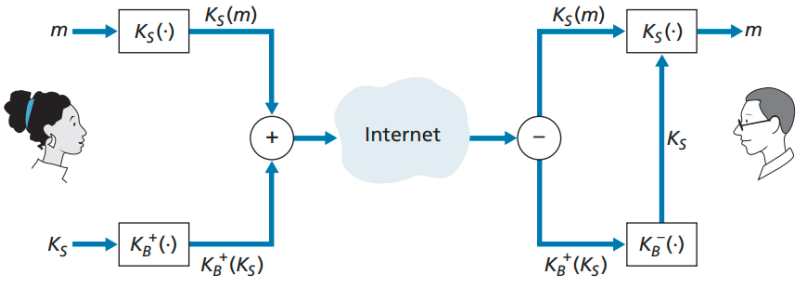


Figure 8.19 ♦ Alice used a symmetric session key, K_S, to send a secret e-mail to Bob

For **full security** (confidentiality + authentication + integrity):

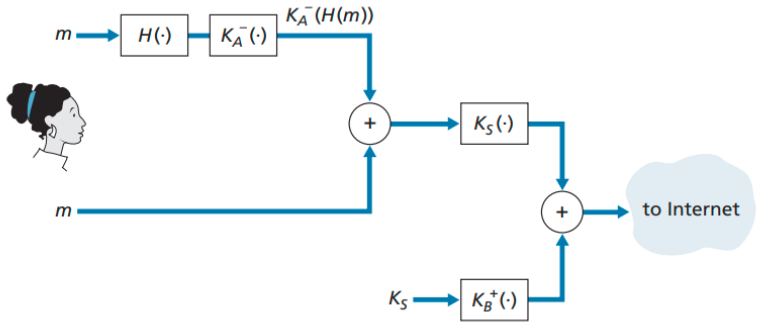


Figure 8.21 ♦ Alice uses symmetric key cyptography, public key cryptography, a hash function, and a digital signature to provide secrecy, sender authentication, and message integrity

TLS (Transport Layer Security)

TLS (successor to SSL) secures TCP connections with **confidentiality, message integrity, and authentication**. It has three phases: handshake, key derivation, and data transfer.

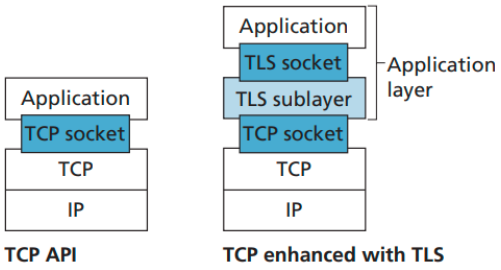


Figure 8.24 ♦ Although TLS technically resides in the application layer, from the developer's perspective it is a transport-layer protocol

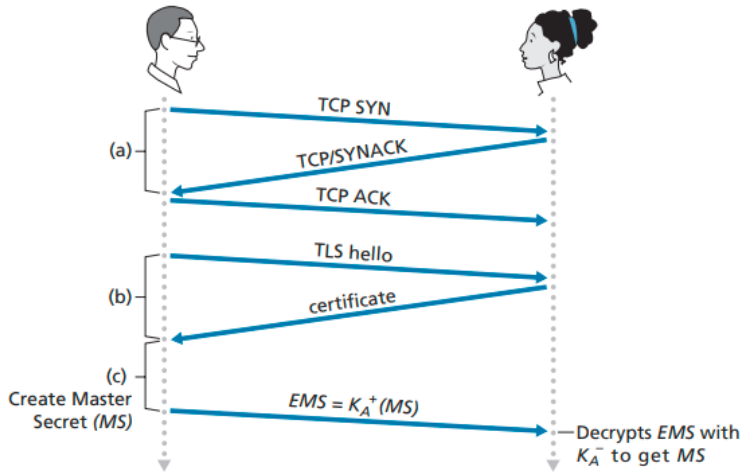


Figure 8.25 ♦ The almost-TLS handshake, beginning with a TCP connection

TLS Handshake steps:

1. **Client Hello:** The client sends its supported cryptographic algorithms and a random **nonce**.
2. **Server Hello + Certificate:** The server selects algorithms, sends its certificate and a server nonce. Includes a hashed/signed version of all prior messages to ensure integrity.
3. **Client Key Exchange:** The client verifies the certificate, extracts the server's public key, generates a **Pre-Master Secret (PMS)**, encrypts it with the server's public key, and sends it.
4. **Key Derivation:** Both sides independently compute the **Master Secret (MS)** from the PMS and nonces using the same key derivation function, producing **four keys**: two session encryption keys and two HMAC keys (one of each for each direction).
5. The client sends an HMAC of all handshake messages.
6. The server sends an HMAC of all handshake messages.

Data transfer: TLS breaks data into records, appends an HMAC (using the HMAC key) for integrity, then encrypts the record + HMAC with the session key. Sequence numbers are included in the hash to prevent deletion/reordering attacks.

TLS record format:



Figure 8.26 ♦ Record format for TLS

When closing: the server must receive the **TLS closure record** before the TCP FIN.

IPSec and VPNs

IPSec provides network-layer security, encrypting the entire payload (including ICMP control messages). **VPNs (Virtual Private Networks)** use IPSec to create a logically private network over the public Internet:

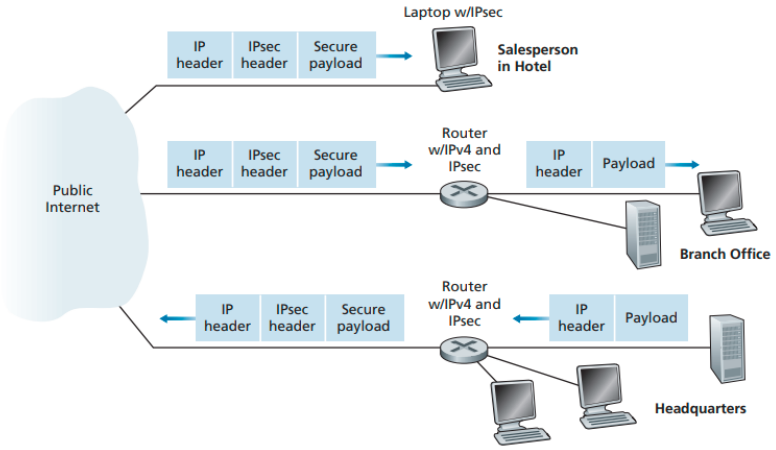


Figure 8.27 ♦ Virtual private network (VPN)

Wireless Security

Wireless channels are inherently easier to attack since messages are broadcast.

WPA3 authentication:

1. The Authentication Server (AS) sends a nonce to the mobile device.
2. The mobile generates its own nonce and derives a **symmetric session key** from both nonces and an initial shared secret. It sends its nonce and an HMAC-signed value back.
3. The AS derives the same session key from the shared secret and both nonces. The key is **never transmitted**: HMAC is used only for verification.
4. The AS provides the session key to the AP.

4G LTE authentication: The mobile attaches to the base station and MME, which requests authentication from the HSS. The HSS derives tokens from the shared secret. The mobile validates the token (confirming it came from the home network), computes a response, and sends it to the MME. The MME verifies and distributes encryption keys to the base station.

8.5: Firewalls

- **Stateless packet filtering**: Examines each packet independently. Drop/forward decisions are based on header info: source/destination IP, TCP/UDP ports, ICMP type, TCP SYN flag, etc.
- **Stateful packet filtering**: Tracks the state of every TCP connection (SYNs, FINs). Determines whether packets “make sense” in context. Includes connection timeouts.
- **Application gateways**: Filter based on application-layer data as well as IP/TCP/UDP fields. Authorized users connect through a gateway server that relays the connection.

Limitations: IP spoofing can bypass filtering; each application needing special treatment requires its own gateway; client software must know how to contact the gateway.

Chapter 9: Multimedia Networking

9.1: Overview

A multimedia application is any application that employs audio or video. Three categories:

- **Streaming stored audio/video** (e.g., Netflix, Spotify).
- **Conversational voice/video-over-IP** (e.g., Zoom, VoIP).
- **Streaming live audio/video** (e.g., live broadcasts).

PCM (Pulse Code Modulation): An analogue signal is sampled at a fixed rate and each sample is quantized. A typical rate is 64 kbps.

The most important performance measure for video is **average throughput**. Conversational applications are **delay-sensitive but loss-tolerant**.

9.2: Video Streaming

Three approaches: UDP streaming, HTTP streaming, and DASH. All make extensive use of **client-side buffering**.

The general pipeline: constant server transmission rate → variable network delay → variable reception rate → **client playout delay (buffering)** → constant playout rate.

UDP Streaming

- The server transmits at a rate matching the video consumption rate.
- No congestion control, the server pushes packets freely.
- A separate control connection (e.g., RTSP) handles commands like pause/resume.
- Disadvantages: requires a media control server, and UDP is often blocked by firewalls.

HTTP Streaming

- The video is stored as a URL and transferred via HTTP (TCP).
- **Prefetching:** TCP's congestion avoidance naturally causes download to run faster than consumption, building up a buffer.
- If the sending rate drops below consumption, the buffer empties and the video freezes.
- Users jumping forward use the **HTTP byte-range header** in a new GET request.
- Early termination wastes buffered data, so systems use moderate-size buffers.

9.3: Voice-over-IP (VoIP)

Most VoIP uses **UDP** because TCP's congestion control introduces too much delay.

Packet jitter: fluctuations in inter-packet arrival times.

Playout Delay Strategies

- **Fixed playout delay:** Each packet has a timestamp t. The receiver plays it at time t + q (a fixed delay). Simple but inflexible.
- **Adaptive playout delay:** The playout delay adjusts at the start of each talk spurt based on estimated network delay and its variance:

$$d_i = (1, u) \times d_{i–1} + u \times (r_i, t_i)$$

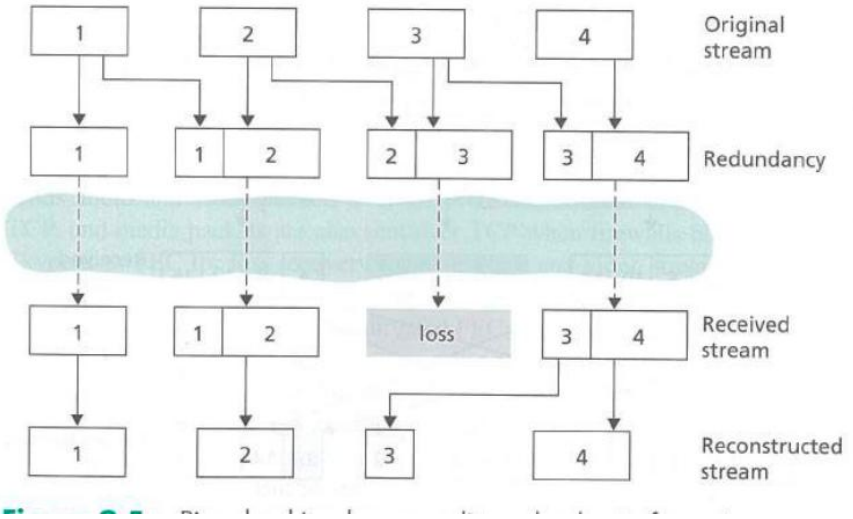
where t_i is the timestamp, r_i is the receive time, d_i is the delay estimate, and u is a smoothing constant.

$$v_i = (1, u) \times v_{i–1} + u \times |r_i, t_i, d_i|$$

The playout time for the first packet in a talk spurt: $p_i = t_i + d_i + K \times v_i$. Subsequent packets in the same spurt play at p_j = t_j + q_i (where q_i = p_i, t_i).

Loss Recovery

- **Forward Error Correction (FEC)**: Two mechanisms:
  - Send a redundant chunk with every n packets (the XOR of all n), forming an (n+1)-block. If one packet is lost, it can be reconstructed.
  - Send a lower-quality redundant copy alongside the primary stream.



- **Interleaving**: Resequence audio before transmission. Spreads the impact of a lost packet across non-adjacent segments, making it less perceptible. Trade-off: increases latency.
- **Error concealment**: Reconstruct lost audio by replaying the previous packet or interpolating between neighbors.

RTP (Real-Time Transport Protocol)

RTP runs on top of UDP and provides structure for real-time data. The RTP header includes:

- **Payload type**: Identifies the audio/video encoding.
- **Sequence number**: For detecting loss and reordering.
- **Timestamp**: For synchronization and playout timing.
- **SSRC (Synchronization Source Identifier)**: A randomly assigned number identifying the stream.

